

A dark blue vertical bar on the left side of the page. A blue arrow points to the right from the bar, containing the date.

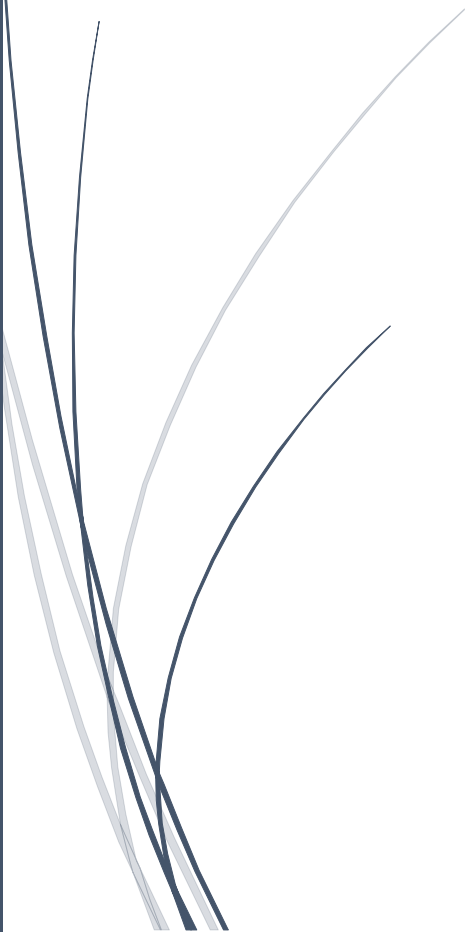
25-5-2026

DOCUMENTACIÓN

Sistemas de gestión de
departamentos del IES Brianda de
Mendoza

GitHub:

https://github.com/OasisSenpai/daw2_proyecto

Several thin, curved lines in dark blue and light grey that sweep upwards from the bottom left towards the center of the page.

David Ramos y Ruben Casañ
2ºDAW PIMD

Contenido

Introducción	4
1. Evaluación	4
1.1. Evaluación inicial.....	4
1.2. Planificación de las revisiones	4
2. Indicadores de calidad.....	5
2.1. Casos de prueba y criterios pass/fail	5
2.2. Batería de procedimientos de pruebas	6
3. Gestión de incidencias.....	6
3.1. Formato del registro de incidencias	7
3.2. Sistematización y temporalización de incidencias.....	7
Flujo de Sistematización (Ciclo de vida)	7
Temporalización (Tiempos de resolución)	7
4. Gestión de cambios	8
4.1. Formato del registro de cambios.....	8
4.2. Formato del análisis del impacto, acciones y decisiones.....	9
5. Implementación y codificación	10
5.1. Diccionario y modelos de datos.....	10
Esquema entidad-relación	10
Modelo relacional y normalización	11
Diseño físico de la BBDD	11
5.2. Backend.....	13
5.3. Frontend	15
5.4. Guía de estilos.....	18
6. Evaluación final	19
6.1. Casos de uso implementados	19
6.2. Arquitectura final del proyecto.....	23
6.3. Cumplimiento de objetivos y plazos	26
6.4 Validación y cumplimiento técnico	28
6.4.1. Procedimiento de participación y evaluación de usuarios (Clientes)	28
6.4.2. Cobertura y Automatización de Pruebas (Testing)	28
6.4.3. Consistencia e Integridad de la Información (BBDD)	29
6.4.4. Cumplimiento de Estándares Web y Compatibilidad.....	29
7. Resultado de las pruebas	30
7.1. Informe de resultados.....	30
7.2 Verificación de funcionalidades	34

8.	Registro de incidencias.....	35
9.	Registro de cambios	39
10.	Despliegue del proyecto.....	41
10.1.	Despliegue en servidor, incluyendo base de datos	41
10.2.	Conexión en cliente	43
11.	Manual de uso.....	43
11.1.	Para usuarios.....	43
11.2.	Para administradores.....	49
12.	Conclusiones	51
12.1.	Síntesis de resultados	51
12.2.	Líneas futuras de trabajo	52
	Conclusión.....	53

Introducción

El presente documento detalla la estrategia de gestión y control de calidad diseñada para el desarrollo de una aplicación de software, liderada por **David Ramos y Ruben Casañ**. El proyecto nace de la necesidad de transformar un sistema rudimentario de gestión de datos basado en archivos Excel independientes en una base de datos relacional robusta y funcional.

Para garantizar el éxito de esta transición y la entrega de un producto óptimo antes del **31 de mayo de 2026**, se ha establecido una hoja de ruta estructurada en fases que abarcan desde el levantamiento de requerimientos hasta el despliegue final. Esta documentación sirve como guía operativa, definiendo no solo los hitos técnicos, sino también los indicadores de calidad, los protocolos de pruebas y los sistemas de gestión de incidencias y cambios necesarios para alcanzar una versión 1.0 estable y eficiente.

1. Evaluación

En este apartado de la documentación vamos a preparar sistemas de gestión y evaluación de diferentes aspectos de nuestra aplicación.

1.1. Evaluación inicial.

Como evaluación inicial partimos de un “sistema” de bases de datos rudimentarios partiendo de Excel de datos con poca relación los cuales modificaremos para convertirlos en una base de datos para su utilización posteriormente en nuestra aplicación.

Después de ver el sistema de trabajo que tenemos en este proyecto hemos decidido utilizar **xtreme programming** como metodología ágil ya que realmente modificaremos el código adaptándonos a los requisitos de Luis a lo largo del desarrollo.

1.2. Planificación de las revisiones

La planificación de las revisiones se irá realizando en base a cambios significativos en nuestra aplicación que puedan afectar a la integridad de la unidad.

Es decir, nuestra versión 0.1 se centrará en la creación de la interfaz, la 0.2 en la conexión con la base de datos y así sucesivamente hasta el pulido final de cara a la 1.0.

Todas estas revisiones se irán decidiendo en un plazo de dos semanas desde el inicio del proyecto para delimitarnos metas de cara a entregar una aplicación totalmente optima y funcional para la fecha acordada.

Usaremos un diagrama de Gantt para esta tarea:

Fase	Tarea Principal	Inicio	Fin
Fase 1	Levantamiento de requerimientos y Arquitectura	01 Mar	14 Mar
Fase 2	Diseño UI/UX y Prototipado	15 Mar	28 Mar
Fase 3	Desarrollo Frontend (Maquetación y lógica)	29 Mar	11 Abr
Fase 4	Desarrollo Backend y Base de Datos	12 Abr	25 Abr
Fase 5	Integración de API y Funcionalidades	26 Abr	09 May
Fase 6	QA (Pruebas), Debugging y Optimización	10 May	23 May
Cierre	Despliegue (Deploy) y Documentación Final	24 May	31 May

2. Indicadores de calidad

Dentro de los indicadores de calidad vamos a medir objetivamente si nuestra aplicación cumple con los estándares necesarios para su lanzamiento.

2.1. Casos de prueba y criterios pass/fail

Usaremos una tabla donde analicemos los diferentes casos de prueba que vayamos implementando e iremos chequeando los criterios entre los que trabajaremos.

ID	Caso de Prueba	Criterio de Éxito (PASS)	Criterio de Fallo (FAIL)
CP-01	Carga de dataset masivo (100k filas).	La tabla se renderiza en < 3 segundos.	Bloqueo del navegador o > 5 segundos.
CP-02	Filtrado combinado.	El resultado es preciso según la lógica booleana aplicada.	No actualiza los datos o muestra registros erróneos.

ID	Caso de Prueba	Criterio de Éxito (PASS)	Criterio de Fallo (FAIL)
CP-03	Exportación a PDF/Excel.	El archivo mantiene el formato y caracteres especiales (ñ, á).	Error en la descarga o archivos corruptos.
CP-04	Persistencia de vistas.	Los filtros guardados se mantienen tras cerrar sesión.	La configuración se pierde al reiniciar la app.

2.2. Batería de procedimientos de pruebas

Se ha elaborado una plantilla que ayudara a controlar los procedimientos de pruebas que surgen a lo largo del proyecto con el fin de asegurarse que ninguna queda sin registrar. Se podrá visualizar cumplimentada en el apartado el anexo y su estructura básica de la plantilla es la siguiente:

ID	Prueba	Pasos	Resultado Esperado	Estado
01	Carga 100k filas	Cargar CSV pesado	Renderizado en < 3s	[]
02	Filtros	Aplicar 2 filtros a la vez	Datos precisos	[]
03	Buscador	Escribir término parcial	Resaltado inmediato	[]
04	Exportar PDF	Clic en botón exportar	Archivo legible (tildes ok)	[]
05	Modo Oscuro	Cambiar tema	Interfaz cambia sin error	[]

3. Gestión de incidencias

El análisis del registro de incidencias permite categorizar los fallos según su criticidad, identificar su origen técnico y determinar el esfuerzo necesario para su resolución, asegurando que los recursos se enfoquen en los errores que comprometen la estabilidad de la versión 1.0.

3.1. Formato del registro de incidencias

ID	Incidencia	Prioridad	Estado	Responsable	Resolución
BUG-01	La tabla se desborda en móvil	Media	Resuelto	David	Ajuste de CSS (overflow)
BUG-02	Exportación PDF falla con "ñ"	Alta	En Proceso	Ruben	Revisión de fuente UTF-8
BUG-03	Error 404 al filtrar por fecha	Crítica	Abierto	Ruben	Pendiente de revisión
BUG-04	El botón "Limpiar" no hace nada	Baja	Pendiente	David	Pendiente

3.2. Sistematización y temporalización de incidencias

Para garantizar que la versión 1.0 sea estable antes del cierre del proyecto (31 de mayo), se establece el siguiente flujo de trabajo y tiempos de respuesta:

Flujo de Sistematización (Ciclo de vida)

Toda incidencia detectada deberá pasar obligatoriamente por los siguientes estados:

1. **Nueva/Abierta:** Detectada y registrada en la tabla.
2. **En Análisis:** El equipo técnico está identificando la causa raíz.
3. **En Desarrollo:** Se está programando la solución.
4. **Validada:** El error ha sido corregido y probado con éxito.
5. **Cerrada:** La solución se ha integrado en la versión principal.

Temporalización (Tiempos de resolución)

Se asignan tiempos máximos para la corrección de errores basándose en la prioridad analizada:

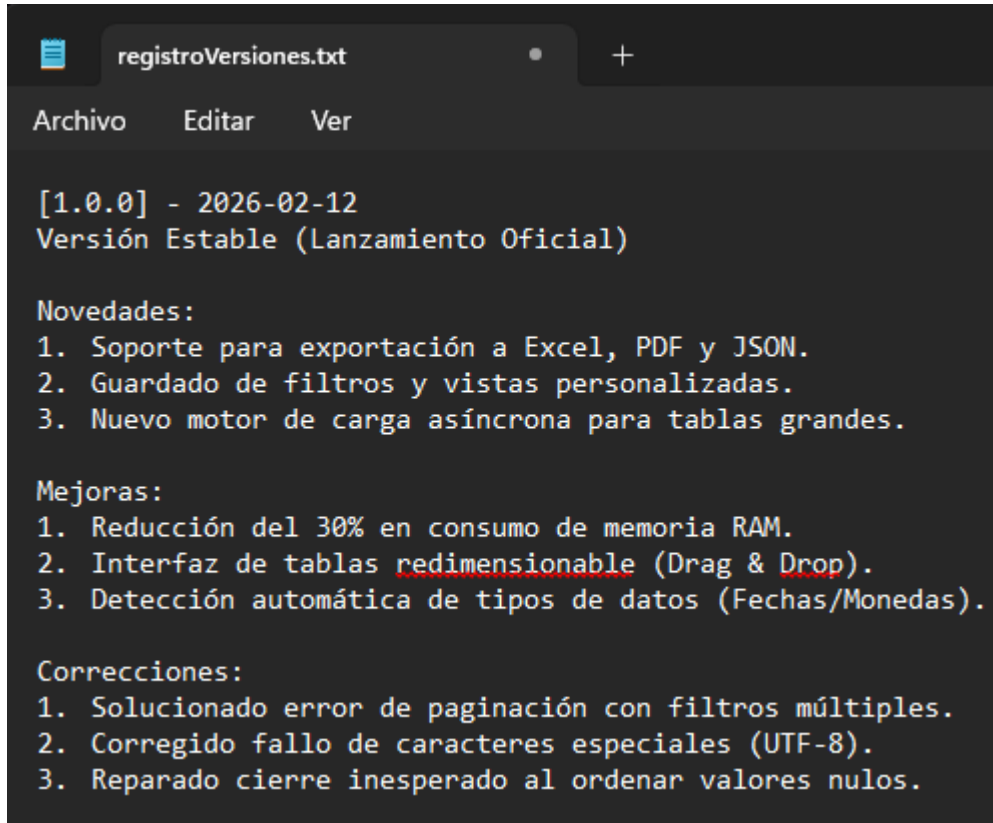
Prioridad	Descripción	Tiempo de Respuesta (SLA)
Crítica	La aplicación no funciona o hay pérdida de datos.	< 12 horas
Alta	Funcionalidad clave (filtros/exportación) falla.	< 24 horas
Media	Errores de interfaz o lentitud puntual.	3 a 5 días hábiles
Baja	Detalles estéticos o sugerencias de mejora.	Próxima versión (v1.1)

4. Gestión de cambios

Dentro de este apartado vamos a crear un sistema para la gestión de cambios a lo largo de las versiones de nuestra aplicación. Para ello hemos decidido crear un archivo .txt para las actualizaciones de las versiones ya que es un formato cómodo y compacto para poder agrupar la información necesaria sobre las modificaciones de nuestra aplicación.

4.1. Formato del registro de cambios

Como ya hemos comentado usaremos un bloc de notas donde iremos actualizando las modificaciones de nuestra aplicación a medida que avance el proyecto y que adjuntaremos en el ANEXO del proyecto.



```
registroVersiones.txt
Archivo  Editar  Ver

[1.0.0] - 2026-02-12
Versión Estable (Lanzamiento Oficial)

Novedades:
1. Soporte para exportación a Excel, PDF y JSON.
2. Guardado de filtros y vistas personalizadas.
3. Nuevo motor de carga asíncrona para tablas grandes.

Mejoras:
1. Reducción del 30% en consumo de memoria RAM.
2. Interfaz de tablas redimensionable (Drag & Drop).
3. Detección automática de tipos de datos (Fechas/Monedas).

Correcciones:
1. Solucionado error de paginación con filtros múltiples.
2. Corregido fallo de caracteres especiales (UTF-8).
3. Reparado cierre inesperado al ordenar valores nulos.
```

Ejemplo de registro de cambios

4.2. Formato del análisis del impacto, acciones y decisiones

En este apartado usaremos una matriz IAD para analizar el impacto de nuestras acciones y/o decisiones dentro del proyecto.

La Matriz IAD (Impacto, Acciones y Decisiones) es una herramienta de gestión de proyectos y desarrollo de software que se utiliza para documentar y controlar los cambios importantes en una aplicación.

Es especialmente útil en el paso de versiones ya que permite justificar técnicamente cada movimiento:

1. Impacto (¿Qué ocurre?)

- Es el análisis de las consecuencias que tendrá un cambio o una nueva funcionalidad.
- Pregunta clave: ¿Qué se rompe, qué cambia o qué mejora con esta actualización?

2. Acciones (¿Cómo lo solucionamos?)

- Son las tareas técnicas o medidas que se toman para ejecutar el cambio o mitigar un impacto negativo.
- Pregunta clave: ¿Qué pasos vamos a dar para que el impacto sea positivo?

3. Decisiones (¿Por qué lo hacemos así?)

- Es la justificación final. Documenta el razonamiento lógico detrás de la solución elegida frente a otras opciones.
- Pregunta clave: ¿Por qué elegimos esta acción y no otra?

Una vez explicado esto, utilizaremos una tabla para la evaluación de estos 3 apartados a lo largo de todo el desarrollo.

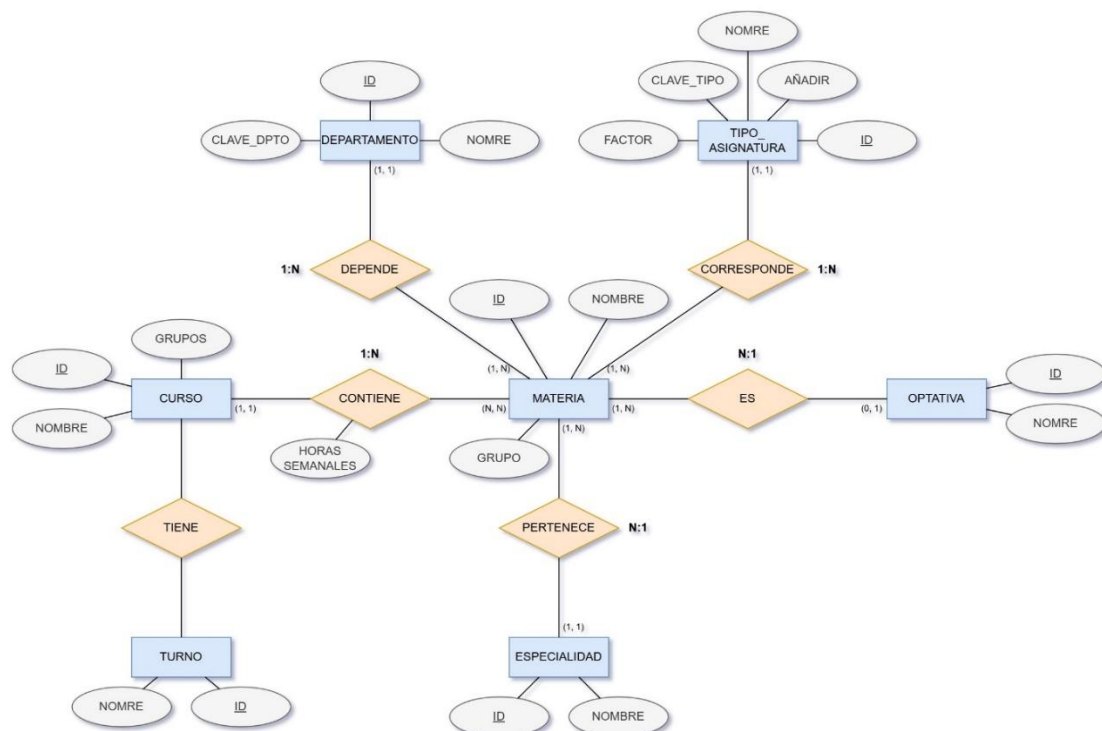
5. Implementación y codificación

En este apartado se detalla la fase de construcción técnica de la aplicación, donde se traslada el diseño conceptual y la arquitectura planificada a un entorno de desarrollo real. Se describirá la estructura de la base de datos que sustituye al sistema de archivos Excel previo, así como la lógica de negocio en el backend y la interfaz de usuario en el frontend. El objetivo es documentar las tecnologías, lenguajes y estándares de codificación utilizados para garantizar un software mantenible, escalable y eficiente.

5.1. Diccionario y modelos de datos

En este apartado vamos a mostrar nuestras tablas desde su diagrama hasta su normalización.

Esquema entidad-relación



Modelo relacional y normalización

CURSO (ID_CURSO, NOMBRE, GRUPOS, TURNOS)

PK: ID_CURSO

En un futuro una materia no podrá pertenecer a varias modalidades, por lo que siempre será una relación N:1.

En un futuro, una materia no podrá ser varias optativas, por lo que nunca será una relación N: M.

MATERIA (ID, TIPO, NOMBRE, ID_CURSO, GRUPO, ESPECIALIDAD, ID_MODALIDAD, ID_OPTATIVA)

PK: ID_MATERIA + CURSO

FK: ID_CURSO (CURSO)

ID_MODALIDAD (MODALIDAD)

ID_OPTATIVA (OPTATIVA)

DEPARTAMENTO (ID, CLAVE_DPTO, NOMBRE)

PK: ID_DEPTO

TIPO_ASIGNATURA (CLAVE_TIPO, NOMBRE, AÑADIR, FACTOR)

PK: CLAVE_TIPO

OPTATIVA (ID, NOMBRE, COMENTARIO)

PK: ID

MODALIDAD (ID, NOMBRE)

PK: ID

ESPECIALIDAD (ID, NOMBRE)

PK: ID

Una materia solo se impartirá en varios cursos, y un curso siempre tendrá varias materias. Por lo que siempre será una relación 1: N.

CURSO_CONTIENE_MATERIA (ID_CURSO, ID_MATERIA, HORAS_SEMANALES)

PK: ID_CURSO + ID_MATERIA

FK: ID_CURSO (CURSO)

ID_MATERIA (MATERIA)

Diseño físico de la BBDD

Cursos

#	Nombre	Tipo de datos	Longitud/Co...	Sin signo	Permitir...	Relle...	Predeterminado	Comentario
1	id	INT	2	☐	☐	☐	Sin valor predet...	
2	nombre	VARCHAR	20	☐	☐	☐	Sin valor predeter...	
3	grupos	INT	1	☐	☐	☐	Sin valor predeter...	
4	turnos	INT	1	☐	☒	☐	NULL	FK: turnos

Departamentos

#	Nombre	Tipo de datos	Longitud/Co...	Sin signo	Permitir...	Relle...	Predeterminado	Comentario
1	id	INT	2	☐	☐	☐	Sin valor predet...	
2	clave_dpto	VARCHAR	5	☐	☐	☐	Sin valor predeter...	
3	nombre	VARCHAR	30	☐	☒	☐	NULL	

Especialidades

#	Nombre	Tipo de datos	Longitud/Co...	Sin signo	Permitir...	Relle...	Predeterminado	Comentario
1	id	INT	2	☐	☐	☐	Sin valor predet...	
2	nombre	VARCHAR	10	☐	☐	☐	Sin valor predeter...	

Materias

#	Nombre	Tipo de datos	Longitud/Co...	Sin signo	Permitir...	Relle...	Predeterminado	Comentario
1	id	INT	2	☐	☐	☐	Sin valor predet...	
2	nombre	VARCHAR	30	☐	☐	☐	Sin valor predeter...	
3	id_curso	INT	2	☐	☐	☐	Sin valor predet...	FK: cursos
4	grupo	VARCHAR	20	☐	☐	☐	Sin valor predeter...	
5	horas_semanal...	INT	2	☐	☒	☐	'0'	
6	id_tipo	INT	2	☐	☒	☐	NULL	FK: tipo_asignatura
7	id_departame...	INT	2	☐	☒	☐	NULL	FK: departamentos
8	id_especialidad	INT	2	☐	☒	☐	NULL	FK: especialidades
9	id_optativa	INT	2	☐	☒	☐	NULL	FK: optativas

Optativas

#	Nombre	Tipo de datos	Longitud/Co...	Sin signo	Permitir...	Relle...	Predeterminado	Comentario
1	id	INT	2	☐	☐	☐	Sin valor predet...	
2	nombre	VARCHAR	100	☐	☐	☐	Sin valor predeter...	

Tipos_asignatura

#	Nombre	Tipo de datos	Longitud/Co...	Sin signo	Permitir...	Relle...	Predeterminado	Comentario
1	id	INT	2	☐	☐	☐	Sin valor predet...	
2	nombre	VARCHAR	20	☐	☐	☐	Sin valor predeter...	
3	clave_tipo	VARCHAR	2	☐	☒	☐	NULL	
4	añadir	INT	11	☐	☐	☐	Sin valor predeter...	
5	factor	DECIMAL	2,1	☐	☐	☐	Sin valor predeter...	

Turnos

#	Nombre	Tipo de datos	Longitud/Co...	Sin signo	Permitir...	Relle...	Predeterminado	Comentario
1	id	INT	1	☐	☐	☐	Sin valor predet...	
2	nombre	VARCHAR	20	☐	☐	☐	Sin valor predeter...	

5.2. Backend

1. El enrutador

Hemos centralizado todo el tráfico en un único archivo que actúa como **Front Controller**: index.php.

Este archivo evalúa la petición (Request) y decide qué hacer basándose en el método HTTP (GET o POST) y en las variables de la URL (\$_GET):

- **Ruta principal (Visualización HTML):** GET /index.php (o simplemente /)
 - **Lógica:** Si la petición es por el método GET y no se solicita generar un CSV, ejecutamos \$controller->main().
 - **Parámetros extra:** Podemos recibir ?tabla=nombre desde el desplegable de navegación. Nuestra vista usará este parámetro para saber qué tabla pintar (Cursos, Materias, etc.). Si no viene, cargamos "cursos" por defecto.
- **Ruta de exportación (Generar CSV):** GET /index.php?generateCSV=generateCSV&tabla=nombre&filtro=busqueda
 - **Lógica:** En nuestro archivo js/functions.js creamos dinámicamente un formulario invisible en el frontend y lo enviamos. index.php detecta estas variables y llama a nuestro método \$controller->generateCSV(\$table, \$filtro).
- **Ruta de Guardado (Sincronizar base de datos):** POST /index.php
 - **Lógica:** Se activa cuando el usuario pulsa el botón "Guardar tabla". Desde el frontend hacemos una petición asíncrona (fetch) enviando un JSON en el cuerpo de la petición.
 - En index.php leemos el cuerpo de la petición con file_get_contents("php://input"), lo decodificamos a un array asociativo y llamamos a \$controller->saveTable(), devolviendo finalmente un JSON con la respuesta (ej: {"mensaje": "Tabla guardada"}).

2. Nuestro Controlador (controllers/Controller.php)

El controlador que hemos diseñado es una clase bastante ligera. Su función principal es servir de intermediario entre la Vista y el Modelo, abstrayendo a index.php de la complejidad de la lógica de negocio. Lo hemos dotado de tres métodos fundamentales:

1. **main():** Es el orquestador de nuestra interfaz gráfica. Instancia la clase View y va concatenando las distintas partes de la página (header(), mainCols(), table(), footer()). Al final devuelve el objeto vista completo que index.php se encarga de imprimir en pantalla (echo).

2. **generateCSV(\$table, \$filtro):**

- Le pide a nuestro Modelo los datos de la tabla y sus columnas.
- Aplica un filtro mediante la función de PHP `array_filter` y `str_contains`, revisando si el texto introducido en el buscador coincide con el de la segunda columna (índice 1).
- Abrimos una salida directa (`php://output`) y usamos la función nativa `fputcsv` para escribir la cabecera y luego iterar por los datos filtrados, creando un archivo descargable al vuelo. Finalmente, cortamos la ejecución con `exit()`.

3. **saveTable(\$table, \$datos):** Simplemente hace de puente, pasando el array 2D de datos directamente a nuestro método `Model::guardarTabla()`.

3. La Lógica del Servidor y Gestión de Datos (models/Model.php)

Aquí es donde reside el núcleo duro de nuestra aplicación. Es donde ocurre la persistencia y manipulación de datos usando la librería `mysqli`. Hemos implementado dos flujos principales que destacan:

A. Flujo de Lectura (`obtenerTabla`, `mostrarTabla`)

Para mostrar la tabla en la web, hemos hecho que nuestro Modelo funcione de manera muy dinámica:

1. Le pedimos a `INFORMATION_SCHEMA.COLUMNS` los nombres reales de las columnas de la tabla.
2. Hacemos un `SELECT * FROM tabla` genérico.
3. El propio Modelo genera un *String* con código HTML (`<table ...>`) donde insertamos cada dato dentro de etiquetas `<td>` a las que les hemos añadido el atributo mágico `contenteditable='true'`. Esto ha sido una decisión clave, porque es lo que permite que el usuario pueda modificar la tabla visualmente como si fuera una hoja de cálculo.

B. Flujo de Sincronización y Guardado (`guardarTabla`)

Este es el flujo más complejo e ingenioso que hemos desarrollado en el servidor. Como nuestro frontend no avisa diciendo "He cambiado esta celda", sino que **envía un array completo con toda la tabla tal cual se ve en pantalla**, el Modelo tiene que deducir qué ha ocurrido (qué se añadió, qué se modificó y qué se borró).

Lo hemos resuelto de la siguiente manera:

1. **Iteración de inserción/actualización:** Recorremos el array que nos llega del frontend. Por cada fila, intentamos hacer un `INSERT INTO tabla (...) VALUES (...)`.

2. Manejo de Excepciones (Try/Catch):

- Si el INSERT funciona, significa que la fila era nueva (añadida visualmente desde el botón "Añadir fila").
- Si salta el error **1062 (Duplicate entry)**, nos indica que el ID de esa fila ya existe en la BBDD. Entonces nuestro catch atrapa el error y asume que el usuario ha modificado una fila existente, lanzando un UPDATE tabla SET col1 = val1... WHERE id = X.
- Si salta el error **1406 (Data too long)** o **1366 (Incorrect value)**, manejamos la excepción para devolver al usuario mensajes de error comprensibles.

3. **Detección de Filas Borradas (El limpiador final):** Una vez hemos procesado todas las filas que vinieron del frontend, nuestra función hace un SELECT * FROM tabla y compara lo que hay actualmente en la base de datos con el array del frontend. Si encuentra una fila en la base de datos que **no está** en el array recibido... ejecutamos un DELETE, asumiendo que el usuario la borró visualmente y guardó los cambios.

Resumen de nuestro ciclo de vida

El usuario ve una hoja de cálculo en HTML (GET). La edita a su gusto gracias a JavaScript nativo. Al pulsar Guardar, nuestro script de JS lee todas las celdas, construye una "fotografía" en JSON de cómo ha quedado la tabla y la manda vía POST. Nuestro Servidor (PHP) recibe esta foto, intenta meter todo como registros nuevos (INSERT), si falla porque ya existen los actualiza (UPDATE), y por último, hace una limpieza borrando de la BBDD todo lo que ya no aparece en la fotografía (DELETE).

5.3. Frontend

1. La Lógica de Renderizado (Server-Side Rendering)

Para nuestro frontend, decidimos no usar un framework de JavaScript (como React o Angular). En su lugar, optamos por **Server-Side Rendering (SSR)**. Esto significa que nuestro servidor PHP construye el HTML completo y lo envía listo al navegador.

La clase encargada de esto es views/View.php. Hemos diseñado la vista acumulando el código HTML en una variable interna de texto (\$this->phtml). Nuestra clase Controller va llamando a los métodos de la vista en un orden específico para construir la página por "piezas":

- **HTML Base:** Generamos las etiquetas estáticas e importamos nuestros estilos (css/styles.css) y **Bootstrap 5** a través de CDN, lo que nos ha ahorrado mucho tiempo diseñando los elementos base como botones y menús.
- **Inyección Dinámica:** Cuando llega el momento de renderizar la tabla, la Vista llama a nuestro Model::mostrarTabla(). Aquí, PHP crea un gigantesco String con

la etiqueta `<table id='tablaDatos'>` e inserta los datos directamente de la base de datos dentro de etiquetas `<td>`.

2. Los Componentes Principales de la Interfaz

Hemos estructurado nuestra pantalla usando lo que parece ser un layout basado en un grid CSS. Dividimos la interfaz en las siguientes grandes "piezas" o funciones dentro de View.php:

1. El header() (Navegación Superior):

- Es la barra superior. Aquí hemos colocado dos menús desplegables (`<select>`) usando el componente form-select de Bootstrap.
- Uno sirve para elegir opciones extra (como generar CSV).
- El otro es un selector de tablas. Le hemos añadido un pequeño truco de JS integrado: `onchange='form.submit()'`. Al cambiar la selección, el formulario hace una petición GET automáticamente para recargar la página con la nueva tabla seleccionada.

2. mainCols() (Layout principal y Barra Lateral):

- Construimos un contenedor `.parent` dividido en zonas (`div1`, `div2`, `div3`).
- **Cabecera de sección (div1, div2):** Colocamos nuestro logo, el título dinámico de la tabla actual y el **Buscador**, un `<input>` de texto que reacciona cada vez que tecleamos algo (`onkeyup`).
- **Barra de Acciones (div3):** Aquí agrupamos los tres botones principales que dan vida a la app: "Añadir fila", "Eliminar fila" y "Guardar tabla (BBDD)".

3. table() (La zona de la Cuadrícula):

- Es el `div4` que envuelve nuestra tabla de datos. El elemento más importante que le hemos puesto a nuestras celdas (`<td>`) es el atributo **`contentEditable='true'`**. Gracias a esto, el navegador permite que el usuario clique y edite el texto de cualquier celda como si esto fuera Excel, sin necesidad de programar campos de texto de manera manual.

4. El footer() (Cierre de Página y Gestión de Eventos):

- Es la sección que da cierre formal a la estructura HTML de la página, imprimiendo las etiquetas finales `</body>` y `</html>`.
- Además, integra un script dinámico que añade un escuchador de eventos (**`change`**) al menú desplegable de opciones (**`#opciones`**). Si este detecta que el usuario ha seleccionado la opción de exportación (**`generateCSV`**), invoca automáticamente a la función JavaScript

prepararEnvioGeneradorCSV() para procesar y descargar la tabla de datos en formato CSV.

3. La Lógica del Cliente (JavaScript)

Para darle interactividad a la interfaz, no quisimos cargar librerías pesadas, así que programamos todo con **JavaScript Puro** dividiéndolo en dos archivos:

A. Filtrado en Tiempo Real (js/buscador.js)

- Implementamos la función `buscarTabla(value)` que se dispara cada vez que el usuario teclea en el input del buscador.
- **¿Cómo lo hicimos?** Iteramos por todas las filas de la tabla (`<tr>`). Leemos el texto de la segunda celda de cada fila (índice 1). Si el texto contiene lo que el usuario ha escrito, dejamos la fila visible; si no, la ocultamos modificando su CSS con `style.display = 'none'`. Así evitamos hacer peticiones al servidor para buscar.

B. Manejo de la Tabla y Peticiones (js/functions.js)

Aquí pusimos las funciones asociadas a nuestros botones de acción:

- **Selección (seleccionarFila):** Al hacer clic en un `<tr>`, le añadimos la clase CSS `.fila-seleccionada`. Hemos metido un detalle genial: detectamos si el usuario pulsa la tecla Control (`event.ctrlKey`) para permitir seleccionar múltiples filas a la vez.
- **Gestión Visual (añadirFila, eliminarFila):**
 - Para añadir, simplemente creamos un elemento `<tr>` nuevo, le añadimos tantas celdas `<td>` como columnas haya, le ponemos el texto "ESCRIBE AQUÍ" y lo insertamos al final de la tabla en el DOM.
 - Para eliminar, buscamos todas las filas que tengan la clase `.fila-seleccionada` y ejecutamos `fila.remove()`, borrándolas de la pantalla.
- **El Guardado Mágico (guardarTabla):** Es nuestra función estrella. Dado que las modificaciones se hacen en el HTML (añadir filas, borrar, editar texto), nuestro servidor no sabe nada. Esta función asíncrona hace un barrido de toda la tabla visual: crea un gran array de arrays donde recopila lo que dice cada celda, y usamos la **API Fetch** (`fetch("index.php", {method: "POST"...})`) para empaquetarlo todo en un JSON y mandárselo al servidor para que él se encargue de actualizar la base de datos real.
- **Exportación tramposa (prepararEnvioGeneradorCSV):** Para descargar el CSV, en lugar de hacer un enlace estático, construimos un `<form>` HTML invisible al vuelo mediante JavaScript, le metemos los parámetros (tabla, filtro de búsqueda) como campos ocultos y forzamos su envío en una pestaña nueva (`target="_blank"`), activando así nuestra ruta de exportación del controlador.

5.4. Guía de estilos

A continuación te presento la guía de estilos que hemos aplicado en el proyecto. Hemos tomado decisiones muy concretas sobre los colores, la tipografía y la disposición (layout) para darle a nuestra aplicación un aspecto único, coherente y amigable, mezclando nuestro CSS personalizado (generado mediante SCSS) con la base de Bootstrap 5.

Aquí tienes el desglose de nuestras decisiones de diseño:

1. Tipografía Global

- **Fuente Principal:** Hemos decidido aplicar una tipografía monoespaciada a nivel global usando font-family: Consolas;. Esto le da a nuestra aplicación un aspecto técnico y estructurado, lo cual encaja perfectamente con el hecho de que es un gestor de base de datos en formato tabla.

2. Paleta de Colores

Hemos jugado con dos paletas muy diferenciadas: tonos azules/violáceos para la estructura general y la barra de herramientas, y tonos verdes naturales para los datos de la tabla.

Estructura y Contenedores:

- **Fondo Principal (body):** Usamos un azul vibrante (#5080ff).
- **Barra Lateral de Acciones (.div3):** Le hemos puesto un azul más claro y suave (#7aa8eb) con texto en blanco para que resalte del fondo pero no compita con la tabla.
- **Botones de Acción:** Utilizamos un tono violeta/índigo (#8c6aff) con texto blanco. Además, les hemos añadido un efecto interactivo (hover) en el que el fondo se vuelve transparente pero mantienen el borde violeta.

La Tabla de Datos: Para la tabla queríamos una estética tipo "hoja de cálculo" pero con un toque fresco, así que apostamos por los verdes:

- **Cabecera de la tabla (thead th):** Verde Oliva Oscuro (#6B8E23) con texto en blanco. Este mismo color lo usamos sutilmente para los bordes de las celdas.
- **Filas Impares:** Verde Medio (#8FBC8F / *MediumSeaGreen*).
- **Filas Pares:** Verde Claro (#98FB98 / *PaleGreen*).
- **Fila Seleccionada (.fila-seleccionada):** Cuando el usuario hace clic en una fila, sobrescribimos su color por un verde menta muy pálido (#d1e7dd) con texto en verde muy oscuro (#0f5132) para indicar claramente qué elemento está activo.

3. Layout y Estructura Visual (CSS Grid)

En lugar de usar la cuadrícula clásica de Bootstrap para la estructura principal, construimos nuestro propio esqueleto usando **CSS Grid** (display: grid con 5 columnas y 6 filas) al que llamamos .parent. Esto nos permitió distribuir los espacios con total precisión:

- **Zonas Redondeadas:** Nos hemos asegurado de evitar los bordes agresivos. Hemos aplicado un border-radius: 20px a casi todos los elementos flotantes: el buscador, la barra lateral de acciones y, lo que es más complejo, al contenedor de la tabla (.div4).
- **La Tabla "Cápsula":** Para que la tabla encajara perfectamente en su contenedor redondeado sin que las esquinas se "salieran", aplicamos border-collapse: separate y fuimos muy minuciosos asignando bordes redondeados (border-radius: 20px) específicamente a la primera celda superior izquierda, a la superior derecha, y a las inferiores de la última fila.
- **Buscador:** El <input> de búsqueda lo hemos destacado dándole fondo blanco puro, también redondeado y con un tamaño de fuente de 16px para facilitar su legibilidad.

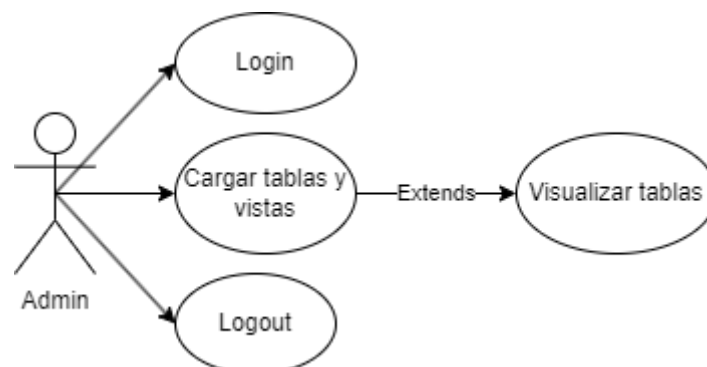
Con esta combinación de CSS Grid para el esqueleto, la tipografía monoespaciada para la lectura de datos, y esta paleta que separa visualmente la "zona de controles" (azules) de la "zona de datos" (verdes), hemos conseguido una interfaz limpia, funcional y con mucha personalidad.

6. Evaluación final

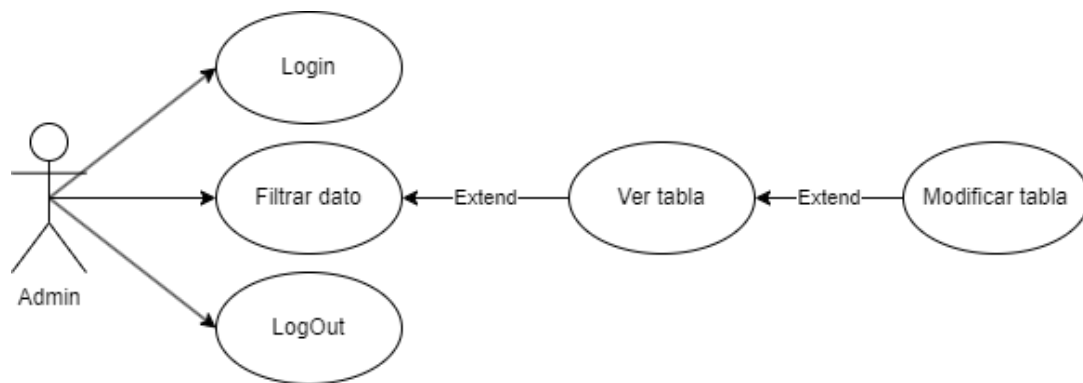
Una vez finalizado el desarrollo, este capítulo presenta el análisis de los resultados obtenidos frente a los objetivos planteados inicialmente. Aquí se verifica el cumplimiento de los casos de uso, la solidez de la arquitectura final y el ajuste a los plazos establecidos en el cronograma. Esta sección actúa como un balance crítico que permite validar si el producto entregado satisface todas las necesidades técnicas y funcionales del cliente. Dentro de los casos de uso tomamos como usuario al administrador ya que es la orientación para la que hemos creado esta aplicación.

6.1. Casos de uso implementados

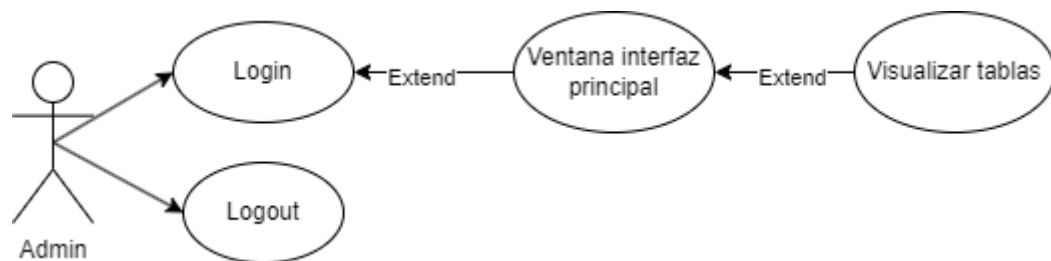
Carga de datos (RF1 Y RF2)



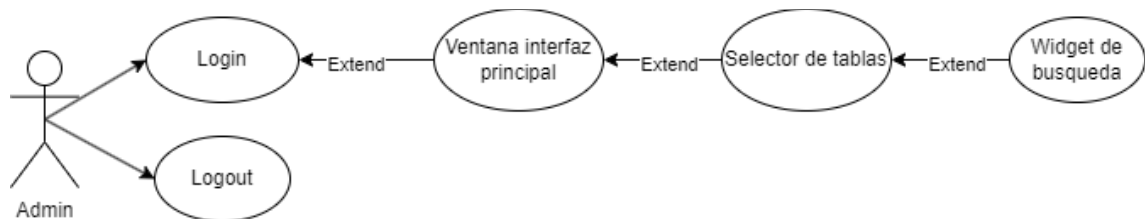
Gestión de datos (RF3 Y RF6)



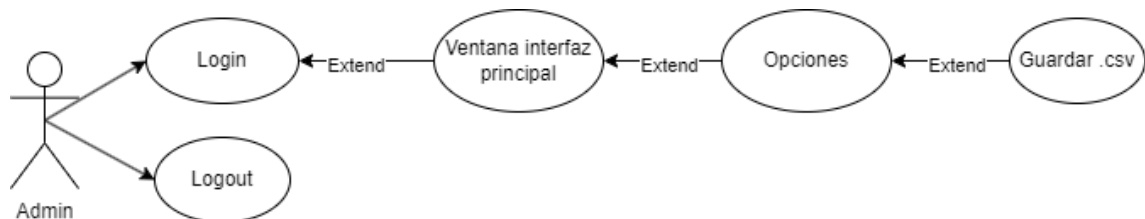
Interfaz gráfica (RF4 Y RF7)



Gestor de búsquedas (RF5)



Conservación de datos (RF8)



Carga de datos

Nombre	Login
Actores:	Administrador
Función:	Da acceso al sistema para poder realizar el resto de acciones.
Referencias:	RF - Todos
Nombre	Cargar la BBDD
Actores:	Administrador
Función:	Al seleccionar una tabla se cargarán los datos de la misma.

Referencias:	RF1 – RF2
--------------	-----------

Gestión de datos

Nombre	Login
Actores:	Administrador
Función:	Da acceso al sistema para poder realizar el resto de acciones.
Referencias:	RF - Todos
Nombre	Filtrar datos
Actores:	Administrador
Función:	Introduce una serie de caracteres en el buscador para filtrar los datos
Referencias:	RF3 – RF6
Nombre	Visualizar la tabla
Actores:	Administrador
Función:	Se cargarán todos los datos necesarios.
Referencias:	RF6
Nombre	Modificar datos
Actores:	Administrador
Función:	Se modificarán los datos pulsando “Añadir fila”, introduces los datos y por último pulsas “Guardar datos”
Referencias:	RF3
Nombre	Logout
Actores:	Administrador
Función:	Salida del sistema.
Referencias:	RF Todos

Interfaz gráfica

Nombre	Login
Actores:	Administrador
Función:	Da acceso al sistema para poder realizar el resto de acciones.
Referencias:	RF - Todos
Nombre	Ventana principal
Actores:	Administrador
Función:	Es donde se encuentra el visualizador de tablas/vistas y los botones de funcionalidad.
Referencias:	RF4
Nombre	Visualizar tablas
Actores:	Administrador
Función:	Muestra la tabla seleccionada.
Referencias:	RF7
Nombre	logout

Actores:	Administrador
Función:	Salida del sistema.
Referencias:	RF Todos

Gestor de búsquedas

Nombre	Login
Actores:	Administrador
Función:	Da acceso al sistema para poder realizar el resto de acciones.
Referencias:	RF - Todos
Nombre	Ventana principal
Actores:	Administrador
Función:	Es donde se encuentra el visualizador de tablas/vistas y los botones de funcionalidad.
Referencias:	RF5
Nombre	Gestor de búsquedas
Actores:	Administrador
Función:	Muestra el dato filtrado por cadena.
Referencias:	RF5
Nombre	logout
Actores:	Administrador
Función:	Salida del sistema.
Referencias:	RF Todos

Conservación de datos

Nombre	Login
Actores:	Administrador
Función:	Da acceso al sistema para poder realizar el resto de acciones.
Referencias:	RF - Todos
Nombre	Ventana principal
Actores:	Administrador
Función:	Es donde se encuentra el visualizador de tablas/vistas y los botones de funcionalidad.
Referencias:	RF8
Nombre	Opciones
Actores:	Administrador
Función:	Menú desplegable
Referencias:	RF8
Nombre	Guardar .csv
Actores:	Administrador
Función:	Se guarda como archivo .csv la tabla seleccionada para exportar la BBDD o crear una copia de seguridad

Referencias:	RF8
--------------	-----

6.2. Arquitectura final del proyecto

Para ilustrar la arquitectura final de nuestro proyecto y cómo se comunican las distintas piezas, he preparado un diagrama visual y una explicación detallada del flujo. Hemos construido una arquitectura **MVC (Modelo-Vista-Controlador)** clásica en el lado del servidor, combinada con un frontend reactivo mediante **JavaScript**.

A continuación, te muestro el diagrama visual de nuestra arquitectura utilizando un esquema que representa los tres flujos principales de comunicación:

Diagrama de secuencia:

C como Cliente (Navegador / JS)

I como index.php (Front Controller)

CT como Controlador (Controller.php)

M como Modelo (Model.php)

V como Vista (View.php)

BD como Base de Datos (MySQL)

1. FLUJO DE CARGA INICIAL (Lectura)

C->>I: GET /index.php?tabla=cursos

I->>CT: Llama a main()

CT->>V: Llama a métodos (header, mainCols...)

CT->>M: mostrarTabla('cursos')

M->>BD: SELECT * FROM cursos

BD-->>M: Devuelve registros

M-->>V: Genera HTML de la tabla con contenteditable

V-->>CT: Devuelve HTML completo

CT-->>I: Devuelve HTML completo

I-->>C: Renderiza la página web

2. FLUJO DE GUARDADO (Escritura Asíncrona)

C->>C: Usuario edita celdas y pulsa "Guardar"

C->>C: JS recopila datos de la tabla en un Array

C->>I: POST /index.php (Fetch API enviando JSON)

I->>CT: Llama a saveTable(tabla, datos)

CT->>M: guardarTabla(tabla, datos)

M->>BD: Intenta INSERT (Nuevos registros)

M->>BD: Si hay duplicados (Error 1062) -> hace UPDATE

M->>BD: DELETE registros que no están en el JSON

BD-->>M: Confirmación

M-->>CT: Array ["mensaje" => "Guardado correcto"]

CT-->>E: Pasa el Array

E-->>C: Responde con echo json_encode()

C->>C: Muestra alert() con el mensaje de éxito

3. FLUJO DE EXPORTACIÓN CSV (Descarga)

C->>C: Usuario pulsa "Exportar tabla"

C->>C: JS crea form invisible y hace submit

C->>I: GET /index.php?generateCSV=1&tabla=x&filtro=y

I->>CT: Llama a generateCSV(tabla, filtro)

CT->>M: obtenerDatosParaCSV(tabla)

M->>BD: SELECT * FROM tabla

BD-->>M: Devuelve registros

M-->>CT: Array de datos puros

CT->>CT: Aplica filtros e inyecta en fputcsv()

CT-->>C: Stream de descarga del archivo .csv

Descripción de nuestra Arquitectura

Como puedes ver en el esquema, hemos diseñado el proyecto para que el flujo de información esté estrictamente controlado. Así es como interactúan nuestras capas:

1. El Cliente (Frontend)

Es la capa de presentación que interactúa con el usuario.

No recargamos la página al hacer ediciones; el usuario escribe directamente sobre las celdas porque les pusimos el atributo contenteditable.

El JavaScript local (functions.js y buscador.js) actúa como un recolector de datos. En lugar de comunicarse constantemente con el servidor con cada tecla que se pulsa, **nuestro cliente espera a que el usuario termine su trabajo**. Al pulsar "Guardar", empaqueta toda la vista de la tabla en un objeto JSON y se la lanza al servidor de un solo golpe usando la API asíncrona fetch.

2. El Front Controller (index.php)

Es la aduana de nuestro servidor. Cualquier petición que hace el cliente (ya sea pedir la página HTML inicial, descargar el CSV o mandar el JSON para guardar) tiene que pasar por aquí. Dependiendo del método HTTP (POST o GET) y de las variables, index.php sabe exactamente a qué método de nuestro Controlador debe llamar.

3. El Controlador (Controller.php)

Funciona como el director de orquesta. Recibe las órdenes de index.php pero **no hace el trabajo pesado**. Si le piden la web, coordina a la View y al Model. Si le piden exportar, pide datos al Model y crea el archivo. Si le piden guardar, le pasa el paquete de datos del frontend directamente al Model.

4. La Vista y el Modelo (View.php y Model.php)

La Vista es un "generador de plantillas" pasivo. Solo sabe concatenar cadenas de texto HTML.

El Modelo es donde pusimos toda nuestra lógica de negocio y reglas. No solo se encarga de hablar con MySQL a través de sentencias SQL preparadas para proteger contra inyecciones (mysqli_real_escape_string), sino que tiene una lógica de sincronización (nuestra función guardarTabla) que compara inteligentemente lo que manda el Frontend con lo que hay en la Base de Datos para decidir si debe ejecutar un INSERT, un UPDATE o un DELETE.

En resumen, hemos logrado construir una arquitectura robusta: el servidor (PHP) tiene la autoridad y hace la conexión con la base de datos de manera segura, mientras que el cliente (JS) ofrece una experiencia fluida e interactiva, comunicándose con el servidor solo en los momentos exactos en los que necesita leer o consolidar información.

6.3. Cumplimiento de objetivos y plazos

Estos eran los objetivos fechados en primera instancia al inicio del proyecto:

Fase	Tarea Principal	Inicio	Fin
Fase 1	Levantamiento de requerimientos y Arquitectura	01 Mar	14 Mar
Fase 2	Diseño UI/UX y Prototipado	15 Mar	28 Mar
Fase 3	Desarrollo Frontend (Maquetación y lógica)	29 Mar	11 Abr
Fase 4	Desarrollo Backend y Base de Datos	12 Abr	25 Abr

Fase	Tarea Principal	Inicio	Fin
Fase 5	Integración y Funcionalidades	26 Abr	09 May
Fase 6	QA (Pruebas), Debugging y Optimización	10 May	23 May
Cierre	Despliegue (Deploy) y Documentación Final	24 May	31 May

Una vez pasada la fase de análisis nos dimos cuenta de que no eran realista y se realizaron modificaciones para adaptarnos a dos factores determinantes: la constante actualización de datos/funcionalidades de la aplicación y los plazos de entrega previstos:

Fase	Tarea Principal	Inicio	Fin
Fase 1	Levantamiento de requerimientos y Arquitectura	01 Mar	14 Mar
Fase 2	Diseño UI/UX y Prototipado	15 Mar	18 Mar
Fase 3	Desarrollo Frontend (Maquetación y lógica)	19 Mar	2 Abr
Fase 4	Desarrollo Backend y Base de Datos	3 Abr	Constante
Fase 5	Integración y Funcionalidades	26 Abr	2 May
Fase 6	QA (Pruebas), Debugging y Optimización	3 May	14 May
Cierre	Despliegue (Deploy) y Documentación Final	15 May	27 May

6.4 Validación y cumplimiento técnico

Para garantizar la estabilidad, la robustez y el mantenimiento a largo plazo del sistema, se han implementado diversas estrategias y protocolos técnicos que aseguran que el software cumple con los estándares exigidos de calidad, integridad de datos y compatibilidad entre navegadores.

6.4.1. Procedimiento de participación y evaluación de usuarios (Clientes)

Para evaluar la idoneidad, usabilidad y rendimiento del Gestor Brianda en un entorno real, se ha establecido un protocolo de pruebas beta en el que ha participado activamente el equipo docente del centro (usuarios finales del pliego de condiciones). Metodología de Evaluación Directa:

* Sesiones de Testeo Guiado: Se concedió acceso al despliegue del servidor en el Aula ATECA a una muestra de profesores para realizar tareas cotidianas (insertar nuevos cursos, modificar claves de departamentos y exportar listados filtrados).

* Instrumento de Recogida de Datos (Cuestionario de Calidad): Tras la interacción con la interfaz estilo Excel, los usuarios cumplimentaron un formulario de evaluación basado en la escala ISO/IEC 25010 (Usabilidad y Eficiencia), evaluando los siguientes indicadores cuantitativos:

Indicador Evaluado	Criterio de Aceptación	Resultado en Pruebas	Estado
Curva de Aprendizaje	El usuario debe ser capaz de editar una celda sin formación previa en < 1 min.	Cumplido. El atributo contenteditable elimina la fricción de formularios tradicionales.	PASS
Robustez ante Errores	El sistema debe avisar de forma comprensible si se introduce un texto excesivamente largo.	Cumplido. La captura del error 1406 muestra una alerta emergente clara en el navegador.	PASS
Eficacia de Filtrado	El buscador en tiempo real debe discriminar registros al vuelo.	Cumplido. El filtrado local mediante manipulación del CSS (display: 'none') es inmediato.	PASS

6.4.2. Cobertura y Automatización de Pruebas (Testing)

El sistema cuenta con un ecosistema dual de pruebas automatizadas que cubre tanto la lógica del servidor (backend) como el comportamiento interactivo del cliente (frontend):

1. Pruebas Unitarias e Integración en el Backend (PHPUnit):

- Se ha configurado una suite en phpunit.xml que analiza de forma aislada e integrada los tres pilares del patrón MVC:
 - ModelTest: Valida la conexión segura a la base de datos, la obtención de esquemas de tablas, y de forma crítica, simula el ciclo de vida completo de sincronización de registros (CRUD completo: INSERT, UPDATE, DELETE).
 - ControllerTest: Verifica que el enrutamiento de peticiones devuelva las instancias correctas de la vista.

- ViewTest: Asegura la generación e integridad de la estructura HTML base y la existencia de elementos interactivos clave (botones de acción, estructura de cabecera).
 - Entorno Seguro de Pruebas: Para evitar la alteración de datos reales en producción, las pruebas de base de datos se ejecutan en un entorno aislado (daw_proyecto-tests) controlado mediante la variable de entorno PHPUNIT_TESTING.
2. Pruebas de Comportamiento en el Frontend (QUnit):
- Ejecutadas a través de js/run_tests.html, esta suite valida el correcto funcionamiento de los scripts JavaScript de la interfaz utilizando fixtures del DOM simulados:
 - buscador.js: Comprobación del motor de filtrado en tiempo real (sensibilidad a mayúsculas, comportamiento con cadenas vacías o términos inexistentes).
 - functions.js: Verificación del aumento/decremento de filas en el DOM, asignación de atributos contentEditable, y el control de selección múltiple (con y sin la tecla Ctrl).

6.4.3. Consistencia e Integridad de la Información (BBDD)

La base de datos es la parte más crítica de la aplicación. Para mitigar fallos parciales o escrituras corruptas, el backend implementa:

- Transacciones Atómicas (ACID): La función de sincronización masiva guardarTabla envuelve todas las consultas de inserción, actualización y borrado dentro de una transacción MySQL (begin_transaction). Si alguna de las operaciones falla, se realiza un rollback inmediato, garantizando que la base de datos nunca quede en un estado inconsistente o a medio guardar.
- Resolución Inteligente de Conflictos (UPSERT): En lugar de realizar costosas consultas previas de existencia para cada registro, el sistema intenta insertar los datos directamente. En caso de colisión de claves primarias (captura del código de error MySQL 1062), el sistema lo intercepta con elegancia y redirige la operación hacia un UPDATE.

6.4.4. Cumplimiento de Estándares Web y Compatibilidad

Para asegurar que la experiencia del usuario sea idéntica independientemente del navegador utilizado:

- Compatibilidad Estricta con Motores Chromium y W3C: Se han depurado prácticas obsoletas de maquetación y control de eventos. Un ejemplo de ello es la eliminación de atributos no estándar como onclick dentro de elementos <option> (un fallo de compatibilidad común en navegadores modernos basados en Chromium como Chrome y Edge), sustituyéndolos por escuchadores de eventos estándar de tipo change sobre las etiquetas <select>.
- Uso de HTML5 Semántico y Accesibilidad: Estructuración de la vista (View.php) con elementos semánticos estándar (<header>, <nav>, <body>) e identificadores únicos (id) consistentes para facilitar la automatización de pruebas y la accesibilidad.
- Estricto reporte de errores en PHP: Uso de la configuración nativa mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT) para forzar que cualquier anomalía de conexión o sintaxis en base de datos actúe como una excepción de PHP controlable, evitando fallos silenciosos.

7. Resultado de las pruebas

La calidad del software se garantiza mediante un proceso de verificación riguroso. En este apartado se exponen los resultados derivados de la ejecución de la batería de pruebas definida en la fase de planificación. Se detallan los informes de éxito y los fallos detectados, asegurando que cada funcionalidad de la aplicación responde correctamente a los criterios de aceptación 'pass/fail' establecidos para la versión 1.0.

Guía para la ejecución de pruebas PHP

Para poder ejecutar la batería de pruebas tras clonar o descargar el repositorio de GitHub, únicamente se deben seguir los siguientes pasos en la consola de comandos de la raíz del proyecto:

- `composer install`
- `vendor/bin/phpunit --coverage-html tests/php/coverage-report` (esto creará un informe interactivo de cobertura de código, requiere tener activo Xdebug)

7.1. Informe de resultados

DOCUMENTACIÓN DE NUESTROS TESTS - PROYECTO MVC

1. Nuestros Tests en PHP

- `ControllerTest`
- `ModelTest`
- `ViewTest`

2. Nuestros Tests en JavaScript

- `test_buscador.js`
- `test_functions.js`

NUESTROS TESTS EN PHP

CONTROLLER TEST Ubicación: `php/ControllerTest.php` Descripción: Este es el conjunto de pruebas que hemos creado para nuestra clase Controller, la cual gestiona la lógica principal de nuestra aplicación.

Método: `testMainReturnsAViewObject()`

- **Nuestro Propósito:** Verificar que el método `main()` de nuestro controlador retorna efectivamente un objeto de tipo View.
- **Qué probamos:** Llamamos al controlador y confirmamos que nos devuelve una instancia de nuestra clase View.
- **Notas importantes:** Hemos notado que este test puede fallar si nuestro modelo intenta conectar a la base de datos sin tener una conexión válida.

MODEL TEST Ubicación: php/ModelTest.php Descripción: Este es nuestro conjunto de pruebas para la clase Model, donde nos aseguramos de que todas las operaciones que hemos programado contra la base de datos funcionen a la perfección.

Método: testModelClassExists()

- Nuestro Propósito: Hacer una verificación muy básica para asegurarnos de que la clase Model que hemos creado realmente existe.

Método: testObtenerTabla()

- Nuestro Propósito: Comprobar que podemos obtener correctamente los datos de una tabla desde nuestra BBDD.
- Qué probamos: Nos conectamos a la BBDD, llamamos a la función para traernos los datos de la tabla "cursos" y verificamos que nos retorna un array con los datos y el número total de columnas sin dar error.

Método: testColocarEncabezadoTabla()

- Nuestro Propósito: Verificar que nuestra función genere el HTML correcto para la cabecera de la tabla.
- Qué probamos: Llamamos a la función y validamos que el HTML generado contenga las etiquetas de filas y encabezados correctas.

Método: testSincronizacionCompletaGuardarTabla() [PRUEBA CRÍTICA - CRUD COMPLETO]

- Nuestro Propósito: Comprobar que el complejo sistema de sincronización de datos que hemos diseñado funciona correctamente en todos los casos (INSERT, UPDATE, DELETE).
- Fases de nuestra prueba:
 1. Limpieza Inicial: Antes de empezar, borramos cualquier rastro de datos de test anteriores.
 2. Inserción (INSERT): Enviamos un paquete de datos con un ID nuevo inventado ('999'). Nuestro método detecta que ese ID no existe en la BD y lo inserta.
 3. Actualización (UPDATE): Simulamos que el usuario envía los mismos datos pero con el nombre modificado. Nuestro código intenta el INSERT, se topa con un error de clave duplicada, y ejecuta nuestro UPDATE como salvavidas.
 4. Borrado (DELETE): Simulamos que el usuario borró la fila enviándole a nuestra función una tabla vacía. Nuestro sistema detecta la ausencia del ID y lo elimina de la BD de forma automática.

Método: testManejoDuplicados()

- Nuestro Propósito: Hacer una prueba muy específica centrada solo en nuestro manejo de registros duplicados.
- Qué probamos: Insertamos manualmente una fila, intentamos guardarla con cambios y verificamos que nuestro UPDATE se dispara de forma correcta.

VIEW TEST Ubicación: `php/ViewTest.php` Descripción: Este es nuestro bloque de pruebas para la clase View, la cual hemos diseñado para construir el HTML de nuestra aplicación.

Método: `testInitialPhtmlIsEmpty()`

- Nuestro Propósito: Asegurarnos de que cuando instanciamos una Vista nueva, empiece completamente limpia, sin texto HTML basura acumulado.

Método: `testHeaderGeneratesHtml()`

- Nuestro Propósito: Comprobar que nuestro método de cabecera construye un HTML válido para abrir el documento, incluyendo nuestro título y scripts.

Método: `testMainColsGeneratesActions()`

- Nuestro Propósito: Verificar que estamos generando los botones de acción principales de nuestra interfaz ("Añadir fila", "Eliminar fila" y "Guardar tabla").

Método: `testFooterClosesBodyAndHtml()`

- Nuestro Propósito: Verificar que nuestro método de pie de página escribe las etiquetas de cierre necesarias para el HTML.

NUESTROS TESTS EN JAVASCRIPT

Para el frontend decidimos utilizar QUnit como framework de testing. Hemos preparado nuestros tests para que se puedan lanzar simplemente abriendo nuestro archivo `js/run_tests.html` en cualquier navegador.

TEST BUSCADOR Ubicación: `js/test_buscador.js` Descripción: Aquí hemos colocado las pruebas para asegurar que nuestro buscador en tiempo real filtra las tablas correctamente.

Método: `buscarTabla` filtra correctamente las filas por contenido

- Nuestro Propósito: Verificar que, a medida que el usuario escribe, las filas irrelevantes se ocultan.
- Casos que probamos:
 - Si buscamos "PHP", nuestra fila de PHP se queda visible y las demás se ocultan.
 - Si buscamos algo inexistente, todas las filas se ocultan.

- Si dejamos el buscador vacío, limpiamos el filtro y todas las filas vuelven a ser visibles.

TEST FUNCTIONS Ubicación: js/test_functions.js Descripción: Este archivo contiene las pruebas completas para todas nuestras funciones de manipulación de la tabla.

Método: contarCeldasTabla()

- Nuestro Propósito: Verificar que estamos contando correctamente el número de columnas leyendo los encabezados.

Método: annadirFila()

- Nuestro Propósito: Verificar que somos capaces de agregar una fila a la tabla dinámicamente usando JS.
- Qué probamos: Comprobamos que el número de filas aumente en 1, que tenga las celdas correctas, y que estas tengan la propiedad para que el usuario pueda escribir en ellas.

Método: seleccionarFila()

- Nuestro Propósito: Verificar que la selección de filas funciona y que hemos implementado bien el soporte para la selección múltiple.
- Casos que probamos:
 - Al hacer clic normal, la fila se selecciona.
 - Al hacer clic de nuevo, se deselecciona.
 - Al hacer clic manteniendo presionada la tecla Ctrl, efectuamos selección múltiple.

Método: eliminarFila()

- Nuestro Propósito: Verificar que nuestra función elimina correctamente del HTML las filas que el usuario ha seleccionado.
- Qué probamos: Simulamos la selección de una fila, ejecutamos el borrado y verificamos que el número total de filas disminuya y que esa fila haya sido fulminada del código de la página.

RESUMEN Y EJECUCIÓN

Cobertura lograda:

- En servidor hemos cubierto satisfactoriamente la creación (INSERT), lectura (SELECT), actualización (UPDATE) y borrado (DELETE), además del manejo de errores.
- En cliente hemos cubierto todas las acciones de manipulación visual del usuario (añadir, seleccionar, eliminar y buscar).

Cómo ejecutar las pruebas:

- Para PHP: Ejecutamos el comando de PHPUnit apuntando a la carpeta "tests/php/". Requiere conexión válida a la base de datos.
- Para JavaScript: Simplemente abrimos el archivo "js/run_tests.html" haciendo doble clic. Utiliza estructuras HTML falsas (fixtures) integradas para hacer las pruebas sin romper la web real.

7.2 Verificación de funcionalidades

1. Carga de Datos (RF1 y RF2)

- **Estado: Cumplido**
- **Descripción Técnica:** El sistema mapea dinámicamente el esquema de la base de datos MariaDB e inyecta los registros en la capa de presentación sin hardcodear las estructuras de las tablas.
- **Verificación:** Validado en el backend mediante la suite automatizada de PHPUnit. Específicamente, el método **ModelTest::testObtenerTabla()** corrobora que la consulta genérica extrae la información en un array bidimensional estructurado, mientras que **ModelTest::testColocarEncabezadoTabla()** verifica que se lean correctamente los metadatos desde INFORMATION_SCHEMA.COLUMNS para generar las etiquetas **<th>** dinámicas.

2. Gestión de Datos (RF3 y RF6)

- **Estado: Cumplido**
- **Descripción Técnica:** El cliente dispone de capacidades completas en el DOM para alterar la estructura de la cuadrícula de datos (añadir y eliminar nodos) y formatear salidas de archivos legibles.
- **Verificación:** Validado en el frontend mediante el framework QUnit en **tests/js/run_tests.html**. El test **annadirFila()** verifica de forma exacta que el número de filas del cuerpo de la tabla (**<tbody>**) se incremente en una unidad, que las nuevas celdas incorporen el atributo nativo **contentEditable="true"** y que nazcan con el string por defecto "ESCRIBE AQUÍ". Por su parte, el test **eliminarFila()** confirma la purga física y la destrucción del nodo del DOM seleccionado.

3. Funcionalidades de la Interfaz Gráfica (UI/UX – RF4 y RF7)

- **Estado: Cumplido**
- **Descripción Técnica:** Interacción reactiva y control visual de estados de la tabla estilo hoja de cálculo (soporte para selección simple y múltiple acumulativa sin recarga de página).
- **Verificación:** Validado en el frontend mediante QUnit en el archivo **test_functions.js**. El caso de prueba **seleccionarFila** simula de forma matemática los eventos del ratón evaluando dos escenarios críticos:

1. Un clic ordinario (con la propiedad **ctrlKey: false**) añade la clase CSS **.fila-seleccionada** al elemento actual y la remueve de cualquier otra fila previa.

2. Un clic combinado (con la propiedad `ctrlKey: true`) puentea la limpieza visual, permitiendo la acumulación síncrona de múltiples registros seleccionados en el DOM para su posterior borrado en lote.

4. Gestor de Búsquedas (RF5)

- **Estado: Cumplido**
- **Descripción Técnica:** Motor de filtrado algorítmico local en tiempo real que discrimina filas por coincidencia de strings en la segunda columna, mitigando la latencia de peticiones repetitivas al servidor.
- **Verificación:** Validado en el frontend mediante la suite **test_buscador.js** de QUnit. Se evalúan secuencialmente cuatro escenarios empíricos simulando la entrada en el input **#buscador**: la búsqueda exacta de un término (ej. "PHP"), la alternancia hacia otro descriptor (ej. "JS"), el comportamiento ante cadenas inexistentes (ocultación total de filas mediante la regla CSS **style.display = 'none'**) y la limpieza del campo (restauración inmediata de la visibilidad de toda la tabla con una cadena vacía "").

5. Conservación de Datos (Persistencia Crítica – RF8)

- **Estado: Cumplido**
- **Descripción Técnica:** Sincronización atómica y consolidación del estado visual del frontend (JSON) sobre el modelo relacional físico en el servidor de base de datos.
- **Verificación:** Validado en el entorno de integración mediante la prueba crítica integral de PHPUnit **ModelTest::testSincronizacionCompletaGuardarTabla()**. El test ejecuta de forma secuencial y aislada en la base de datos de pruebas (**daw_proyecto-tests**) las tres fases del algoritmo CRUD síncrono:
 1. Fase de Inserción: Comprueba que un nuevo registro (ID **999**) se cree con éxito.
 2. Fase de Actualización (UPSERT): Simula la edición del registro y valida que el bloque try-catch capture correctamente el código de excepción de clave duplicada MySQL **1062** para conmutar la consulta hacia un **UPDATE** de forma transparente.
 3. Fase de Limpieza: Envía un dataset vacío para verificar que el servidor detecte de forma proactiva la ausencia del ID **999** en el JSON y ejecute un **DELETE** automático en la base de datos para sincronizar los estados.

8. Registro de incidencias

Durante los meses de desarrollo, fuimos documentando y solucionando las siguientes incidencias técnicas:

INCIDENCIA #001 (05/02/2026)

- **Problema:** Error "Class not found" en index.php. La web se quedaba en blanco al intentar cargar.

- **Causa:** Problemas con las rutas relativas al usar include para el archivo del Controlador.
- **Solución:** Cambiamos include por require_once asegurando la ruta correcta (controllers/Controller.php).
- **Estado:** Resuelto

INCIDENCIA #002 (15/02/2026)

- **Problema:** Error 404 / Variables vacías al guardar. El servidor PHP no recibía los datos al pulsar "Guardar".
- **Causa:** La API fetch de JS enviaba los datos como JSON crudo, por lo que el array tradicional \$_POST de PHP llegaba vacío.
- **Solución:** Modificamos el enrutador para leer directamente con file_get_contents("php://input") y decodificar el JSON.
- **Estado:** Resuelto

INCIDENCIA #003 (22/02/2026)

- **Problema:** Error general de conexión a la BBDD. Aparecían advertencias (warnings) de conexión en varias funciones del Modelo a la vez.
- **Causa:** Las credenciales de MySQL estaban repetidas de forma rígida (hardcodeadas) en cada método.
- **Solución:** Centralizamos la conexión creando un archivo de configuración externo llamado bbdd.php.
- **Estado:** Resuelto

INCIDENCIA #004 (02/03/2026)

- **Problema:** Descuadre del Layout (CSS Grid). Si la tabla tenía muchas filas, empujaba el panel de herramientas y el footer fuera de la pantalla.
- **Causa:** El contenedor .div4 de la cuadrícula Grid crecía infinitamente junto con el contenido.
- **Solución:** Añadimos la regla overflow-y: scroll al contenedor .div4 para mantener la cuadrícula fija e incluir una barra de desplazamiento interna.
- **Estado:** Resuelto

INCIDENCIA #005 (08/03/2026)

- **Problema:** El buscador ocultaba filas para siempre. Al buscar un texto y luego borrarlo, las filas originales no reaparecían.
- **Causa:** El script de JavaScript eliminaba físicamente los nodos del HTML (remove()) del DOM, perdiendo los datos.

- **Solución:** Cambiamos la lógica visual en buscador.js. Ahora no borra los elementos, solo alterna su estilo CSS (`style.display = 'none'`).
- **Estado:** Resuelto

INCIDENCIA #006 (12/03/2026)

- **Problema:** Las celdas deformaban la tabla entera. Al escribir textos muy largos en el modo de edición de celdas, la tabla se estiraba sin límite.
- **Causa:** No había restricción de comportamiento para los desbordamientos de texto continuo en el CSS.
- **Solución:** Se ajustaron las propiedades de la tabla (`border-collapse`) y se definió un comportamiento de ancho predecible en `styles.css`.
- **Estado:** Resuelto

INCIDENCIA #007 (18/03/2026)

- **Problema:** Los bordes redondeados no se aplicaban a la cabecera. Las esquinas superiores de la tabla seguían siendo cuadradas.
- **Causa:** El CSS general de `border-radius: 20px` no afecta a los elementos internos de la tabla (`<th>` y `<td>`) que tocan los bordes.
- **Solución:** Añadimos reglas CSS específicas (`first-child` y `last-child`) apuntando a las esquinas exactas (ej. `border-top-left-radius`).
- **Estado:** Resuelto

INCIDENCIA #008 (25/03/2026)

- **Problema:** Aviso *Undefined array key "filtro"* al intentar exportar un archivo CSV.
- **Causa:** Si el usuario exportaba sin haber escrito nada en el buscador, la variable no se creaba y PHP lanzaba un Warning.
- **Solución:** Se aplicó el operador de fusión nula (`?? ''`) en `index.php` para asignar automáticamente un texto vacío si no venía filtro en la URL.
- **Estado:** Resuelto

INCIDENCIA #009 (05/04/2026)

- **Problema:** El selector de tablas no respondía. El usuario seleccionaba "Departamentos" pero la pantalla no cambiaba.
- **Causa:** Faltaba un botón de envío ("`submit`") en el formulario de navegación HTML para ejecutar la acción.
- **Solución:** En lugar de añadir un botón extra, insertamos el evento `onchange='form.submit()'` directamente en la etiqueta `<select>` para recargar la página automáticamente.

- **Estado:** Resuelto

INCIDENCIA #010 (10/04/2026)

- **Problema:** Texto con comillas rompía el guardado (Fallo de seguridad por inyección SQL).
- **Causa:** Las comillas insertadas por el usuario en las celdas cerraban prematuramente las sentencias de texto SQL en el servidor PHP.
- **Solución:** Aplicamos la función nativa `mysqli_real_escape_string()` a todos los campos en el backend antes de montar las consultas INSERT o UPDATE.
- **Estado:** Resuelto

INCIDENCIA #011 (22/04/2026)

- **Problema:** Fallo crítico al modificar filas (No se guardaban los cambios editados).
- **Causa:** Al mandar la tabla entera, MySQL lanzaba el *Error 1062 (Duplicate entry)* al intentar insertar un ID de fila que lógicamente ya existía en la BBDD.
- **Solución:** Implementamos un bloque try/catch en Model.php. Ahora capturamos el error 1062 para ejecutar dinámicamente una consulta UPDATE como plan B.
- **Estado:** Resuelto

INCIDENCIA #012 (28/04/2026)

- **Problema:** Textos largos se truncaban en silencio sin avisar al usuario.
- **Causa:** MySQL cortaba la información automáticamente si superaba el límite de tamaño (VARCHAR) del campo. Se generaba el error interno 1406 (*Data too long*) pero la app no lo mostraba.
- **Solución:** Capturamos explícitamente el código de excepción 1406 y ahora devolvemos un JSON de error para que JS lance una alerta visual (`alert()`) al usuario.
- **Estado:** Resuelto

INCIDENCIA #013 (02/05/2026)

- **Problema:** La eliminación múltiple de filas no funcionaba (solo se borraba una).
- **Causa:** El selector de JavaScript solo quitaba la clase "fila-seleccionada" de los elementos previos sin contemplar combinaciones de teclas de selección múltiple.
- **Solución:** Añadimos la lectura del evento de teclado (`event.ctrlKey`) en JS para permitir acumular la selección en varias filas simultáneas.
- **Estado:** Resuelto

INCIDENCIA #014 (05/05/2026)

- **Problema:** La exportación a CSV devolvía todo el código fuente de la web o HTML basura.
- **Causa:** La vista principal (HTML) se imprimía antes de empezar a procesar los datos del CSV, corrompiendo el formato del archivo.
- **Solución:** Forzamos la descarga asignando las cabeceras Content-Type: text/csv desde el servidor y añadimos la instrucción `exit()` inmediatamente después de llamar a `fputcsv()`.
- **Estado:** Resuelto

INCIDENCIA #015 (12/05/2026)

- **Problema:** Colapso total del sistema por dejar un campo ID vacío.
- **Causa:** MySQL arrojaba el *Error 1366 (Incorrect integer value)* al recibir un texto vacío en un campo numérico principal durante el intento de guardado.
- **Solución:** Añadimos una validación adicional en el bloque *catch* para el código 1366. Ahora forzamos un `continue`; en el bucle PHP para ignorar la inserción defectuosa silenciosamente y evitar que caiga el resto del proceso.
- **Estado:** Resuelto

9. Registro de cambios

[2026-05-23]

Refactorizado (Refactor)

- ****Sincronización y Guardado de Tablas:**** Rediseño completo de la función ``guardarTabla`` en la clase ``Model`` para usar transacciones MySQL y automatizar la detección de ``INSERT``, ``UPDATE`` y ``DELETE``. Incluye validación de duplicados, modularización en funciones privadas y un reporte detallado con el recuento de operaciones procesadas (``48fb164``).

Corregido (Fixes)

- ****Compatibilidad de exportación CSV en Chromium:**** Corrección en ``View.php`` sustituyendo el uso de ``onclick`` no estándar en etiquetas `<option>` por un escuchador del evento ``change`` en el selector general de la función ``footer()`` (``6bdd5e6``).

- ****Base de datos de columnas:**** Especificación de la base de datos correcta al obtener la estructura de columnas de las tablas en ``Model.php`` (``27a13aa``).

[2026-05-14]

Documentación (Docs)

- Añadida documentación detallada del proyecto. (Merge PR #2) (`387d50f`, `374d9a8`).

[2026-05-12]

Añadido (Added)

- **Sistema de tablas:** Implementado el sistema completo de selección y eliminación de filas en la tabla (`1a6fdf9`).
- **Tests (PHPUnit y QUnit):**
 - Creación del archivo `phpunit.xml` para la configuración de PHPUnit.
 - Agregado archivo HTML de pruebas QUnit para pruebas de JavaScript.
 - Implementación de pruebas para `buscador.js` (incluyendo funcionalidad de filtrado).
 - Agregadas pruebas para `functions.js` (conteo de celdas, adición, selección y eliminación de filas).
 - Creación de pruebas PHPUnit para las clases `Controller`, `Model` y `View` para garantizar el correcto manejo de datos y lógica MVC (`8c97253`, `c5773a7`). (Merge PR #1)

Mejorado (Changed)

- **Exportación CSV:** Mejora en la funcionalidad de exportación a CSV para tener en cuenta los filtros aplicados (`1a6fdf9`).

[2026-04-26]

Añadido (Added)

- **Guardado de tablas:** Implementada la funcionalidad `saveTable` y actualizada la vista para poder guardar el estado de las tablas mediante peticiones AJAX (`aee9be9`).

[2026-04-18]

Refactorizado (Refactor)

- **Estilos y SCSS:** Reestructuración completa de los estilos CSS. Se eliminaron archivos sin uso y se integró SCSS para mejorar la mantenibilidad visual del proyecto (`ad3ddbc`).

[2026-01-24]

Refactorizado (Refactor)

- **Arquitectura:** Conversión de la estructura del proyecto a un patrón de diseño **MVC** (Modelo-Vista-Controlador) (`'88d79c7'`).

Añadido (Added)

- **Exportación CSV:** Al exportar a CSV ahora se aplica correctamente el filtro de búsqueda de la interfaz (`'88d79c7'`).

[2026-01-04]

Inicial (Initial Commit)

- **Primer lanzamiento:** Se implementó la visualización inicial de las tablas obtenidas desde la base de datos (`'74c6c12'`).
- **Buscador:** El filtro de búsqueda por nombre es totalmente funcional en esta versión (`'74c6c12'`).

10. Despliegue del proyecto

El despliegue representa la transición del entorno de desarrollo al entorno de producción. En esta sección se describe el procedimiento técnico seguido para poner la aplicación a disposición de los usuarios finales, incluyendo la configuración del servidor, la migración de la base de datos y los protocolos de conexión necesarios. Se detallan las herramientas utilizadas para asegurar que el sistema sea accesible de forma estable y segura a través de la red.

10.1. Despliegue en servidor, incluyendo base de datos

Este documento detalla los pasos exactos que debemos seguir para llevar nuestro proyecto desde nuestro entorno de desarrollo local y ponerlo en producción en los servidores del Aula ATECA.

Fase 1: Preparación del Paquete en Local

Antes de mover nada al servidor del instituto, necesitamos preparar nuestros archivos:

1. Exportar la Base de Datos:

- Abrimos nuestro gestor local (como phpMyAdmin, DBeaver o MySQL Workbench).
- Seleccionamos nuestra base de datos completa.
- La exportamos generando un archivo `.sql` (por ejemplo, `bd_gestor_brianda.sql`). Asegurándonos de marcar la opción "Añadir

sentencia DROP TABLE / VIEW" para que sobrescriba tablas antiguas si ya existieran.

2. Empaquetar el Código Fuente:

- Comprimimos toda la carpeta daw2_proyecto-main en un archivo .zip.
- **Atención:** Podemos excluir la carpeta tests/ y los archivos de pruebas (phpunit.xml, composer.json), ya que no son necesarios para que la aplicación funcione en el servidor final y por motivos de seguridad es mejor no subirlos.

Fase 2: Configuración de la Base de Datos en el Aula ATECA

Una vez conectados al entorno del Aula ATECA:

1. Acceder al Gestor de Bases de Datos:

- Entramos al panel de administración de MySQL que nos proporcione el Aula ATECA (generalmente phpMyAdmin o a través de consola).

2. Importar la Estructura y los Datos:

- Creamos una nueva base de datos vacía (si no nos han asignado una previamente).
- Vamos a la pestaña "Importar", seleccionamos nuestro archivo bd_gestor_brianda.sql y ejecutamos la importación.

3. Tomar nota de las Credenciales:

- Apuntamos cuidadosamente el **Servidor (Host)**, el **Usuario**, la **Contraseña** y el **Nombre de la Base de Datos** específicos que nos ha asignado el Aula ATECA para este proyecto.

Fase 3: Subida de Archivos y Configuración

1. Transferir el Código:

- Subimos nuestro archivo .zip al directorio público del servidor del Aula ATECA (habitualmente la carpeta /var/www/html/ o public_html/) usando un cliente FTP (como FileZilla) o el administrador de archivos del servidor.
- Descomprimos el archivo allí mismo.

2. Modificar la Conexión (El paso más crítico):

- En el servidor, abrimos nuestro archivo de configuración: models/bbdd.php.
- Modificamos las variables de conexión que usábamos en casa ("localhost", "root", "") y las sustituimos por las credenciales reales del servidor del Aula ATECA que anotamos en la Fase 2.
- Guardamos los cambios en el servidor.

Fase 4: Permisos y Verificación Final

1. Revisar Permisos de Carpetas (Opcional pero recomendado):

- Nos aseguramos de que los archivos PHP tengan los permisos de lectura adecuados (generalmente 644) y las carpetas permisos de acceso (755) para que el servidor Apache/Nginx del Aula ATECA pueda leerlos sin lanzar un *Error 403 (Forbidden)*.

2. Testeo de Humo (Smoke Test):

- Abrimos el navegador web y accedemos a la URL pública que nos haya asignado el Aula ATECA (ej. http://ateca.instituto.edu/nuestro_proyecto).
- Comprobamos que el diseño se ve correctamente (esto verifica que el servidor lee nuestra carpeta css/).
- Realizamos una prueba creando una fila nueva, guardando, y exportando a CSV para garantizar que los permisos de base de datos (INSERT/UPDATE/DELETE) y las cabeceras HTTP de descarga funcionan en producción exactamente igual que en local.

10.2. Conexión en cliente

El proyecto ha sido desplegado exitosamente y se encuentra operativo en los servidores del Aula ATECA. A continuación, proporcionamos los datos necesarios para que el equipo docente pueda acceder y evaluar la herramienta en un entorno de producción real.

Enlace de Acceso

Puede acceder a la versión final de la aplicación haciendo clic en el siguiente enlace: [\[http://url-del-aula-ateca.es/ruta-a-vuestro-proyecto\]](http://url-del-aula-ateca.es/ruta-a-vuestro-proyecto)

11. Manual de uso

Con el fin de facilitar la adopción del sistema y asegurar su correcto funcionamiento, se han elaborado guías detalladas para los distintos perfiles de usuario. Este capítulo se divide en un manual operativo para usuarios finales, enfocado en las tareas cotidianas de gestión, y un manual técnico para administradores, centrado en la gestión de permisos, mantenimiento de datos y configuraciones avanzadas del sistema.

11.1. Para usuarios

MANUAL DE USUARIO: GESTOR DE DATOS BRIANDA

Bienvenido/a al Gestor Brianda. Esta aplicación web ha sido diseñada para que puedas administrar la información de la base de datos de manera tan sencilla e intuitiva como si estuvieras utilizando una hoja de cálculo tipo Excel.

A continuación, te explicamos paso a paso cómo utilizar todas las funcionalidades del sistema:

1. NAVEGACIÓN Y BÚSQUEDA

Cómo cambiar de tabla

En la parte superior izquierda de la pantalla encontrarás un menú desplegable con el texto "Tablas". Al hacer clic en él, verás la lista de todos los apartados disponibles (Cursos, Departamentos, Materias, etc.).

- Simplemente selecciona la tabla que deseas visualizar. La página se actualizará automáticamente y te mostrará los datos correspondientes.

The screenshot shows the 'Cursos' interface. On the left, there is a sidebar with 'Opciones' and 'Tablas' dropdowns. The 'Tablas' dropdown is open, showing a list of tables: Cursos, Departamentos, Especialidades, Horas departamentos (vista), Materias, Materias (vista), Optativas, Tipos de asignatura, and Turnos. A red arrow points to the 'Tablas' dropdown. Below the sidebar, there are three buttons: 'Añadir fila', 'Eliminar fila', and 'Guardar tabla (BBDD)'. The main area displays a table with the following data:

	nombre	grupos	turnos
1	1º ESO	4	1
2	2º ESO	4	1
3	3º ESO	3	1
4	4º ESO	3	1
5	1º Bachillerato	5	1
6	2º Bachillerato	4	1
7	1º GH SPR	3	1
8	2º GH SPR	3	1
9	1º GS ASTR	2	1
10	2º GS ASTR	2	1
11	1º GS DAM	2	1
12	2º GS DAM	2	1
13	1º GH ACOM	1	1
14	2º GH ACOM	1	1
15	1º GSCINT	2	1
16	2º GSCINT	2	1
17	1º GSTYL	2	1
18	2º GSTYL	2	1
19	1º GB Admon	1	1

Cómo buscar y filtrar datos

En la parte superior derecha de la interfaz encontrarás una barra de búsqueda.

- Al escribir cualquier texto en ella, la tabla se filtrará automáticamente en tiempo real (por ejemplo, si escribes "PHP", solo se mostrarán las filas que contengan esa palabra).
- Si borras el texto, volverán a aparecer todas las filas originales.

The screenshot shows the 'Cursos' interface with the search bar highlighted by a red rectangle. The search bar contains the text 'esd'. The table below shows only the first four rows, which are ESO courses:

id	nombre	grupos	turnos
1	1º ESO	4	1
2	2º ESO	4	1
3	3º ESO	3	1
4	4º ESO	3	1

2. EDICIÓN DE DATOS (ESTILO EXCEL)

El Gestor Brianda permite editar la información directamente sobre la pantalla, sin necesidad de abrir ventanas secundarias ni formularios complicados.

Cómo modificar una celda existente

1. Haz clic directamente sobre cualquier celda de la tabla (el texto que quieres cambiar).
2. Borra o escribe el nuevo contenido con tu teclado.
3. **Importante:** Estos cambios son temporales y solo se ven en tu pantalla. Para que los cambios sean definitivos, debes guardarlos (ver sección 4).



id	clave_dpto	nombre
1	ByG	Biología y Geología
2	EDF	Educación Física
3	Fra	Francés
4	GvH	Geografía e Historia
5	LcYL	Lengua Castellana y Literatura
6	Ing	Inglés
7	Mat	Matemáticas
8	Mus	Música

Cómo añadir una nueva fila

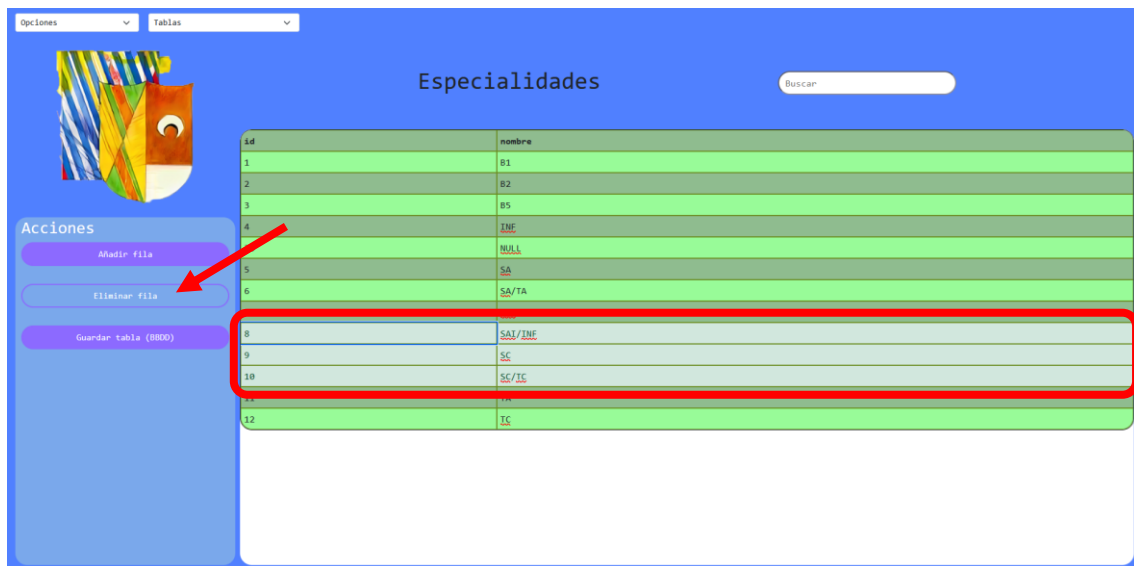
1. En el panel lateral derecho (sección "Acciones"), haz clic en el botón "Añadir fila".
2. Verás que aparece una fila nueva al final de la tabla con el texto "ESCRIBE AQUÍ" en cada celda.
3. Haz clic sobre cada "ESCRIBE AQUÍ" y sustitúyelo por los datos reales.
4. Recuerda rellenar correctamente la primera columna (el ID) si es necesario, ya que es la clave que identifica el registro.



id	nombre
1	B1
2	B2
3	B3
4	INF
10	NULL
5	SA
6	SA/TA
7	SAI
8	SAI/INF
9	SC
10	SC/TC
11	TA
ESCRIBE AQUÍ	ESCRIBE AQUÍ

Cómo eliminar filas

1. Haz clic sobre la fila que deseas borrar. Verás que cambiará de color para indicarte que está seleccionada.
2. Si te has equivocado, vuelve a hacer clic en ella para deseleccionarla.
3. **Selección múltiple:** Si necesitas borrar varias filas a la vez, mantén pulsada la tecla **Control (Ctrl)** de tu teclado mientras vas haciendo clic en las distintas filas.
4. Una vez tengas la fila (o filas) seleccionadas, haz clic en el botón **"Eliminar fila"** en el panel de Acciones de la derecha. Las filas desaparecerán de tu pantalla.

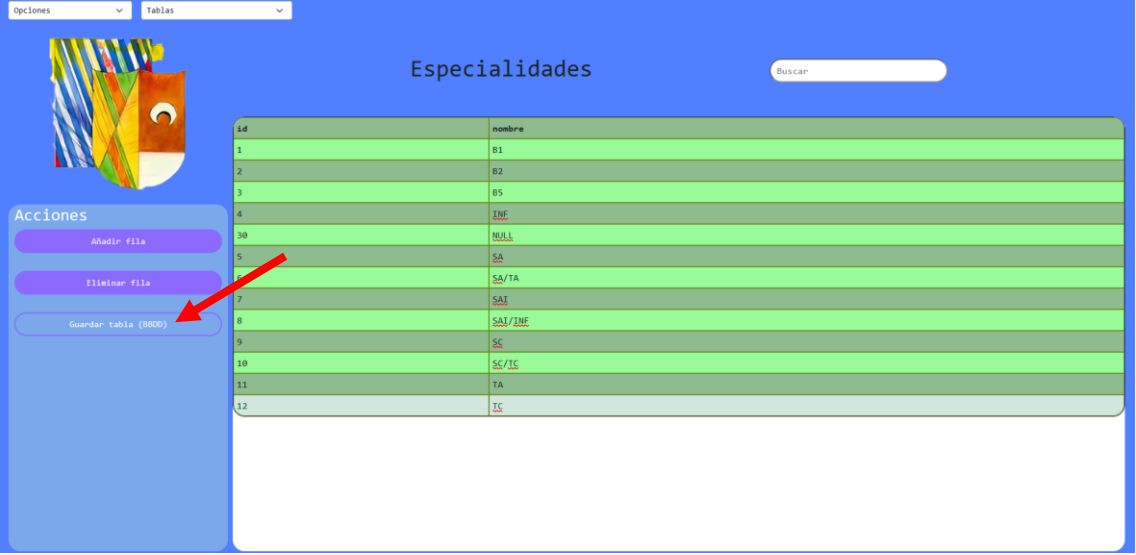


3. GUARDAR LOS CAMBIOS EN LA BASE DE DATOS

¡Paso Crítico! Añadir filas, borrar registros o editar el texto de las celdas **no modifica la base de datos de forma automática**. Esto te permite equivocarte y corregir errores sin miedo.

Una vez que la tabla tenga exactamente el aspecto y los datos que tú quieres que tenga de forma oficial:

1. Dirígete al panel lateral de Acciones.
2. Haz clic en el botón **"Guardar tabla (BBDD)"**.
3. El sistema procesará todos los cambios a la vez y te mostrará un mensaje de confirmación cuando haya terminado. A partir de ese momento, los cambios son definitivos.

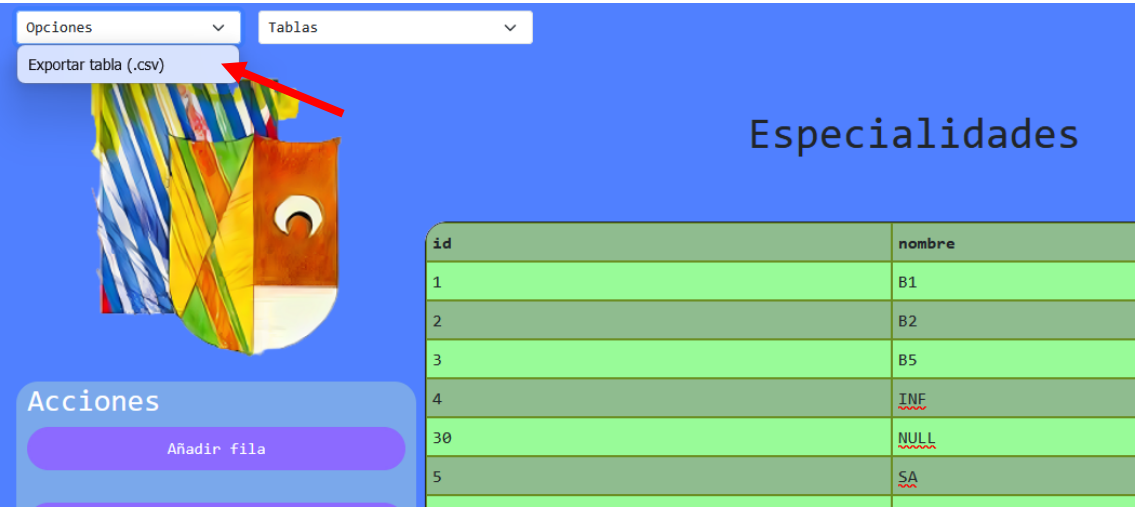


id	nombre
1	B1
2	B2
3	B5
4	INF
30	NULL
5	SA
7	SA/TA
8	SAI
8	SAI/INF
9	SC
10	SC/IC
11	TA
12	IC

4. EXPORTAR DATOS A EXCEL (CSV)

Si necesitas descargar la información para llevarte un informe o mandarla por correo, puedes descargar la tabla en formato CSV (compatible con Excel).

1. Sitúate en la tabla que quieres exportar.
2. Si solo quieres exportar una parte de la tabla, utiliza el **Buscador** para filtrar los datos primero. El archivo descargable solo contendrá lo que estés viendo en pantalla en ese momento.
3. Ve a la parte superior izquierda y haz clic en el menú desplegable llamado **"Opciones"**.
4. Selecciona **"Exportar tabla (.csv)"**.
5. Se descargará automáticamente un archivo en tu ordenador con la información de la tabla.



id	nombre
1	B1
2	B2
3	B5
4	INF
30	NULL
5	SA

11.2. Para administradores

MANUAL DEL ADMINISTRADOR: GESTOR DE DATOS BRIANDA

Este documento está dirigido al personal técnico y a los administradores del sistema. Explica el funcionamiento interno de la aplicación, los requisitos para su mantenimiento y cómo escalar la herramienta (añadir nuevas tablas o vistas).

1. ARQUITECTURA DEL SISTEMA

La aplicación está construida utilizando un patrón MVC (Modelo-Vista-Controlador) puro en PHP para el backend, y Vanilla JavaScript (JS nativo sin frameworks como React o Angular) para el frontend.

- **Frontend (Cliente):** La interfaz se renderiza vía Server-Side Rendering (SSR) desde PHP. Se utiliza Bootstrap 5 (vía CDN) para la base de estilos y Grid CSS puro para el esqueleto. Las celdas de las tablas se hacen editables nativamente usando el atributo HTML `contenteditable="true"`.
- **Enrutador (Front Controller):** Todas las peticiones, tanto GET como POST, pasan exclusivamente por el archivo `index.php`.
- **Backend (Servidor):** El archivo `Controller.php` orquesta las llamadas. El trabajo pesado de base de datos se encuentra en `models/Model.php`, utilizando la extensión `mysqli`.

2. GESTIÓN DE LA BASE DE DATOS Y CONEXIÓN

Configuración de la conexión

Los parámetros de conexión a la base de datos MySQL (host, usuario, contraseña y nombre de la base de datos) se encuentran centralizados. Para modificar el entorno (por ejemplo, pasar de Local a Producción), debe editarse el archivo: `models/bbdd.php`

El Sistema de Sincronización (CRUD Automático)

Es vital que el administrador comprenda cómo funciona la función `guardarTabla()` del Modelo, ya que difiere de los formularios web tradicionales. El sistema no envía peticiones individuales por cada celda modificada. En su lugar, envía un JSON con una "fotografía completa" de la tabla.

El servidor procesa esta fotografía en 3 fases:

1. **Intento de Inserción (INSERT):** Recorre el JSON e intenta insertar cada fila.
2. **Manejo de Duplicados (UPDATE):** Si la clave primaria ya existe, MySQL devuelve el **Error 1062 (Duplicate entry)**. El código captura (catch) este error y ejecuta un `UPDATE` para actualizar los campos.

3. **Barrido de Limpieza (DELETE):** Finalmente, hace una consulta a la BD. Si encuentra un registro en MySQL que ya no está presente en el JSON recibido, asume que fue borrado en el frontend y ejecuta un DELETE automático.

REGLA DE ORO PARA EL ESQUEMA DE BBDD: Para que el sistema de actualización y borrado funcione correctamente, **la Clave Primaria (ID) de todas las tablas debe ser obligatoriamente la primera columna.** El modelo utiliza el índice 0 del array para construir las sentencias WHERE id = X.

3. CÓMO ESCALAR LA APLICACIÓN (AÑADIR NUEVAS TABLAS)

El Gestor Brianda ha sido diseñado para ser altamente dinámico. El Modelo (Model.php) lee las columnas en tiempo real utilizando INFORMATION_SCHEMA.COLUMNS, lo que significa que la aplicación se adapta sola a la estructura de la tabla.

Pasos para dar de alta una nueva tabla en el sistema:

1. Crea la tabla o vista en la base de datos MySQL de forma habitual. Asegúrate de que la primera columna sea el identificador único (ID).
2. Abre el archivo views/View.php.
3. Busca la función header() y localiza el formulario con el selector de tablas (<select name='tabla' id='tabla'...>).
4. Añade una nueva etiqueta <option> con el nombre exacto de tu tabla en MySQL.
Ejemplo: <option value='alumnos_matriculados'>Alumnos Matriculados</option>

¡Eso es todo! No necesitas crear nuevos modelos ni controladores. Al seleccionar esa opción en la web, el sistema leerá los encabezados automáticamente y permitirá su edición, guardado y exportación en CSV.

4. RESOLUCIÓN DE PROBLEMAS COMUNES (TROUBLESHOOTING)

- **Problema:** Al pulsar "Guardar tabla", el sistema devuelve un mensaje de error mencionando problemas de espacio ("Data too long").
 - **Causa:** El usuario ha introducido texto en una celda que excede el límite de caracteres (VARCHAR) configurado en MySQL para esa columna específica. El error 1406 es capturado por el sistema.
 - **Solución:** Ampliar la longitud del campo en MySQL o indicar al usuario que resuma la información.
- **Problema:** Una tabla no carga o no se muestran los datos.

- **Causa:** Generalmente ocurre si el usuario de la base de datos no tiene permisos de lectura sobre INFORMATION_SCHEMA, ya que el sistema lo requiere para generar las cabeceras HTML dinámicamente.

5. MANTENIMIENTO Y EJECUCIÓN DE TESTS

El sistema cuenta con una cobertura de pruebas automáticas para asegurar la integridad de la base de datos y la interfaz en futuras actualizaciones.

Para ejecutar las pruebas del Servidor (Backend): Asegúrese de tener Composer instalado y las dependencias cargadas. Abra la terminal en la raíz del proyecto y ejecute: `vendor/bin/phpunit tests/php/` (Nota: Estas pruebas realizarán operaciones reales de INSERT, UPDATE y DELETE usando identificadores altos como el 999. Es necesaria una conexión a la BD activa).

Para ejecutar las pruebas del Cliente (Frontend): Abra el archivo `tests/js/run_tests.html` directamente en cualquier navegador web. El framework QUnit ejecutará los tests sobre un DOM simulado y le mostrará los resultados en pantalla.

12. Conclusiones

Como cierre de la documentación, se presenta una síntesis de los hitos alcanzados durante el desarrollo del proyecto. En este apartado se reflexiona sobre los retos superados, se evalúa el impacto de la solución tecnológica implementada y se proponen futuras líneas de trabajo que permitan la evolución y mejora continua de la aplicación en versiones posteriores.

12.1. Síntesis de resultados

Durante el desarrollo del Gestor Brianda, hemos superado los objetivos iniciales logrando una aplicación rápida, escalable y muy fácil de usar. Nuestros principales hitos técnicos son:

1. **Arquitectura MVC Propia:** Hemos construido todo el sistema backend (Modelo-Vista-Controlador) en PHP puro, sin depender de frameworks externos pesados, logrando un código ligero y eficiente.
2. **Interfaz "Estilo Excel":** Eliminamos los tediosos formularios web. Gracias a JavaScript nativo y al atributo HTML `contenteditable`, el usuario puede editar, añadir y borrar datos haciendo clic directamente sobre la tabla, de forma súper intuitiva.
3. **Sincronización Inteligente de Datos:** Desarrollamos un algoritmo de guardado (CRUD) que compara un "pantallazo" de la tabla actual con la base de datos, ejecutando automáticamente las inserciones, borrados y actualizaciones (aprovechando el error 1062 de MySQL).

4. **Escalabilidad Automática:** El sistema lee la estructura de la base de datos en tiempo real. Añadir una nueva tabla al gestor no requiere programar nada nuevo, solo añadir una línea de código en el menú desplegable.
5. **Calidad Garantizada:** Hemos asegurado la robustez del proyecto implementando una doble suite de pruebas automáticas: PHPUnit para el servidor y QUnit para el comportamiento de la interfaz visual.

12.2. Líneas futuras de trabajo

Aunque el Gestor Brianda ha alcanzado un alto nivel de funcionalidad, hemos identificado varias áreas de mejora y funcionalidades avanzadas que nos gustaría implementar en futuras versiones del proyecto:

1. **Sistema de Autenticación y Roles de Usuario:** Actualmente, la herramienta es de acceso libre. El siguiente paso fundamental es integrar un panel de *Login* con encriptación de contraseñas. Esto nos permitirá crear roles: un "Administrador" que pueda editar y borrar datos, y un "Invitado" que solo tenga permisos de visualización o exportación.
2. **Paginación de Resultados (Carga Perezosa):** El sistema actual carga y envía toda la tabla de golpe. Si la base de datos creciera a decenas de miles de registros, podría ralentizar el navegador. Implementaremos un sistema de paginación o *Lazy Loading* (cargar a medida que se hace scroll) para mantener el rendimiento impecable sin importar el tamaño de los datos.
3. **Reconocimiento de Claves Foráneas (Desplegables en celdas):** Ahora mismo, todas las celdas permiten escribir texto libre. En el futuro, queremos que el sistema detecte si una columna es una Clave Foránea (por ejemplo, el ID de un Departamento). En ese caso, en lugar de dejar al usuario escribir un número a ciegas, la celda se convertirá automáticamente en un menú desplegable con los nombres de los departamentos disponibles.
4. **Importación masiva mediante CSV:** Ya contamos con una funcionalidad muy sólida para exportar datos a CSV. La evolución natural es desarrollar el camino inverso: permitir al usuario subir un archivo de Excel/CSV para poblar la base de datos de forma masiva en cuestión de segundos.
5. **Historial de Cambios (Deshacer/Rehacer):** Dado que nuestra interfaz imita la experiencia de una hoja de cálculo, queremos mejorar el JavaScript del frontend para implementar combinaciones de teclado clásicas como *Control+Z*. Esto permitirá al usuario revertir ediciones accidentales antes de pulsar el botón de guardar.
6. **Validación de Datos en Tiempo Real:** Añadiremos validadores visuales para que, si un usuario intenta escribir letras en una columna destinada exclusivamente a

números, la celda se marque en rojo al instante, avisando del error antes de que intente enviar los datos al servidor.

Conclusión

A través de la implementación de metodologías ágiles y herramientas de control como el **diagrama de Gantt** y la **matriz IAD (Impacto, Acciones y Decisiones)**, se ha consolidado un marco de trabajo que minimiza la incertidumbre y maximiza la trazabilidad del desarrollo. El uso de indicadores de calidad objetivos (pass/fail) y una jerarquización estricta de incidencias basada en tiempos de respuesta (SLA) asegura que los recursos se enfoquen en la estabilidad crítica del sistema.

En definitiva, la sistematización de los procesos aquí descrita permite que la evolución desde la versión 0.1 hasta el lanzamiento oficial sea un proceso controlado, documentado y capaz de adaptarse a los cambios técnicos mediante un registro de versiones transparente. Este enfoque garantiza que la aplicación final no solo cumpla con los requisitos funcionales, sino que ofrezca una experiencia de usuario optimizada y técnica superior.